

Arbeitsexposé

KI-gestützte 3D-Rekonstruktion linearer Infrastruktur: Evaluierung einer Docker-basierten Scan-to-BIM Pipeline mittels SAM 3 und Gaussian Splatting

Bachelor- und Projektarbeit

17. Juli 2026

1 Einleitung und Forschungsfrage

Die präzise Erfassung schwer zu modellierender, extrem dünner Objekte (wie Stromerkabel in Baugraben) stellt für klassische photogrammetrische Verfahren (SfM) eine große Herausforderung dar. Ziel dieser Arbeit ist die Entwicklung und Evaluierung eines "Scan-to-BIM"-Workflows, der die Vorzüge der 2D-KI-Segmentierung (Segment Anything Model 3) mit modernsten 3D-Rekonstruktionsmethoden (3D Gaussian Splatting via SuGaR) verbindet.

Wie aktuelle Forschungen belegen, erzeugen 3D Gaussian Splatting Algorithmen durch mathematische Wahrscheinlichkeitsfunktionen hochgradig detailreiche, fotorealistische Visualisierungen bei geringer Rechenlast. Sie werden zunehmend im Ingenieurwesen und zur Generierung von Orthofotos (z.B. Integration in ArcGIS Pro) eingesetzt. Jedoch fehlt 3DGS-Modellen die inhärente Georeferenzierung. Die zentrale Forschungsfrage lautet daher: *Lässt sich die 3D-Rekonstruktion linearer Infrastruktur durch eine vorgelagerte 2D-Segmentierung und ein objektspezifisches 3DGS-Training (Segment-then-Splat) so optimieren und georeferenzieren, dass strenge ingenieurstechnische Toleranzen eingehalten werden?*

2 Praxisbezug und Erfolgskriterien

Die Pipeline wird direkt für den Anwendungsfall der TenneT (Übertragungsnetzbetreiber) konzipiert.

- **Geometrische Toleranz:** Eine Abweichung von maximal ± 10 cm in Lage und Höhe gegenüber der realen Kabeltrasse.
- **Georeferenzierung & GIS-Integration:** Das finale 3D-Modell muss präzise in einem UTM-Koordinatensystem vorliegen, um den direkten Import in Geoinformationssysteme (z.B. ArcGIS Pro) zu gewährleisten.
- **Wirtschaftliche Evaluierung:** Eine Kostengegenüberstellung des Drohnen- und Serverrechenaufwands im Vergleich zur klassischen, terrestrischen Vermessung.

3 Methodik und Architektur (Workflow v3)

Der Ansatz basiert auf einer klaren Trennung von 2D-Bildverarbeitung und 3D-Meshing.

3.1 Software-Repositories

- **SAM 3 (Meta)**: Zuständig für Concept-Prompting und Video-Tracking.
- **COLMAP**: Generierung der Kameraposen auf Basis der *unmaskierten* Originalbilder. Die Pipeline nutzt ausschließlich das schnelle *Structure from Motion* (SfM) von COLMAP. Die rechenintensive dichte Rekonstruktion (MVS) wird vollständig übersprungen.
- **Segment-then-Splat (STS)**: Übernimmt das objektspezifische 3DGS-Training (Lu et al., NeurIPS 2025). STS teilt die initialen COLMAP-Gaussians anhand der SAM 3 Masken in objektspezifische Sets auf und trainiert diese mit einem dedizierten Object-specific Loss ($\mathcal{L}_{obj}$), ohne die Ground-Truth-Bilder zu manipulieren. Das Ergebnis ist ein vollständiges 3DGS-Modell, in dem jeder Gaussian eine Object-ID trägt. SAM 3 ersetzt hierbei das intern vorgesehene AutoSeg-SAM2.
- **SuGaR**: Surface extraction with Gaussian Splatting. SuGaR übernimmt einen 3DGS-Checkpoint, richtet die Gaussians mittels geometrischer Regularisierung (insbesondere Depth-Normal Consistency) an der Oberfläche aus und extrahiert über Poisson Surface Reconstruction ein Mesh. Für objektspezifische Eingaben wird SuGaR um maskenkonsistente RGB-, Normalen- und Texturverluste erweitert, damit die Bildsupervision dieselbe semantische Objektdefinition wie STS verwendet.
- **DGtal**: Open-Source Geometrie-Bibliothek zur finalen Centerline-Extraktion. Sie wandelt die Oberfläche des Kabel-Sub-Meshes in eine räumliche 1D-Kurve (Centerline bzw. Scheitelachse mit lokalen Koordinaten) um.

3.2 Pipeline-Struktur: Temporale Konsistenz vs. SAHI

Für die Erkennung extrem dünner Kabel in hochauflösenden 4K-Bildern bietet sich theoretisch *Slicing Aided Hyper Inference* (SAHI) an. SAHI zerschneidet das Bild in kleinere Teilbereiche (Patches), wodurch verhindert wird, dass dünne Kabel beim Downscaling auf die native Modellauflösung von SAM 3 "verschluckt" werden.

Dennoch wurde der primäre Fokus für diese Pipeline bewusst auf den **SAM 3 Video Predictor** gelegt. Der methodische Nachteil von SAHI ist die fehlende temporale Konsistenz: Da SAHI jedes Einzelbild isoliert betrachtet, "flackern" die erzeugten Masken von Frame zu Frame, wenn sich Lichteinfall oder Blickwinkel leicht ändern. 3D Gaussian Splatting erfordert jedoch zwingend formstabile, konsistente Masken über alle Kameraperspektiven hinweg, da sonst fehlerhafte Artefakte in den 3D-Raum projiziert werden. Der SAM 3 Video Predictor löst dieses Problem durch ein Memory-Modul, welches die Kabel-Pixel über die Zeit hinweg verfolgt und dadurch flackerfreie, temporär konsistente Masken liefert. SAHI dient in dieser Pipeline lediglich als Fallback-Route für spezifische Bildausschnitte, in denen das temporale Tracking abreißen sollte.

4 Docker-Infrastruktur und Orchestrierung

Ein Kernproblem moderner KI-Pipelines sind tiefgreifende Versionskonflikte zwischen den benötigten Nvidia-Grafikkartentreibern (CUDA). Dieses Problem wird durch eine **docker-compose** Architektur vollständig gelöst. Die Pipeline isoliert die Anwendungen in voneinander getrennte Container, während alle erzeugten Daten über **Docker Bind Mounts (Volumes)** zentral und persistent auf dem Host-System (Windows/Linux) gespeichert werden.

Um die Automatisierung zu maximieren, wird die Pipeline über ein zentrales Master-Skript (*One-Click-Orchestration*) gesteuert. Dieses Skript stößt die Container sequenziell an, muss jedoch für manuelle Arbeitsschritte pausieren.

Autopilot-Modus für unterbrechungsfreie Verarbeitung:

Um den wissenschaftlichen Workflow anwenderfreundlich zu gestalten und manuelle Interaktionen auf ein Minimum zu reduzieren, wurde ein optionaler **Autopilot-Modus** in die Orchestrierungskonfiguration integriert. Direkt nach der Eingabe des Semantik-Prompts (z.B. "cable") kann dieser aktiviert werden. **Wirkungsweise:** Alle nachgelagerten Parameterentscheidungen werden vollautomatisch mit vordefinierten, wissenschaftlich erprobten Standardwerten belegt. **Automatisierte Parameter:** Das FFMPEG-Pre-Processing erzwingt im Autopilot ein komprimiertes Arbeitsvideo im optimalen Kompromiss von 1920×1080 Pixeln bei 10FPS, einem Constant Rate Factor (CRF) von 23 und dem *x264-Preset medium*. Das STS-Training wird auf standardmäßig 7000 Gesamt-Optimierungsiterationen (bzw. 5000 Iterationen in Stage 2) mit klassischer Speicherallokation festgelegt. **Anwendervorteil:** Bis auf den zwingenden physikalischen Messschritt (Breakpoint für das GCP-Point-Picking in CloudCompare) ist kein Benutzereingriff mehr erforderlich. Die Pipeline steuert alle fünf Microservices automatisiert an, führt den neuen Teilmodell-Wechsler (Best-Model-Resolver) selbständig aus und exportiert die georeferenzierten GIS-Daten fix und fertig in das Zielverzeichnis.

Es kommen fünf dedizierte Container zum Einsatz:

- **Container A (Vorverarbeitung):** Beinhaltet PyTorch 2.x und SAM 3. Benötigt das Nvidia Container Toolkit (CUDA ≥ 12.6).
- **Container B (SfM):** Ein offizielles NVIDIA-Image für COLMAP (CUDA 11.8 oder 12.1). *Nach Ausführung dieses Containers pausiert das Master-Skript automatisch (Breakpoint). Der Nutzer öffnet CloudCompare auf dem Host-System, pickt die GCPs und speichert die Transformationsmatrix. Anschließend wird das Skript mit "Enter" fortgesetzt.*
- **Container C (Object-specific 3DGS):** Beinhaltet Segment-then-Splat (STS). Empfängt die SAM 3 Masken und die COLMAP-Kameraposen und führt das objektspezifische 3DGS-Training mit dem nativen Object-specific Loss ($\mathcal{L}_{obj}$) durch. Benötigt CUDA 12.1 (PyTorch 2.3.1). SAM 3 ersetzt hierbei das intern vorgesehene AutoSeg-SAM2 als Masken-Frontend.
- **Zwischenschritt 3.5 (Kabel-Isolierung & Qualitätsfilterung):** Das Skript `filter_cable_pc.py` erzeugt zusätzlich zur unveränderten Full-Scene-Punktwolke die echt gefilterte Objektwolke `point_cloud_filtered.ply`. Es selektiert die konfigurierte

Ziel-Objekt-ID (im aktuellen Versuchsdatensatz `obj_id_m == 0`) und kann Opazitäts- sowie Farbschwellen anwenden. Eine zweite Kopie, `point_cloud_filtered_opacity999999.ply`, übernimmt dieselben verbliebenen Datensätze und setzt ihre Opazitätslogs für den geometrieorientierten, maskenkonsistenten SuGaR-Eingang einheitlich auf $\alpha = 0,999999$. Diese Überschreibung ist eine Initialisierungspolitik und keine physikalische Aussage über die Transparenz des Objekts. Die getrennte Ablage verhindert, dass ein Objekt-Checkpoint versehentlich als unmaskierte Full-Scene-SuGaR-Baseline dient.

- **Container D (Meshing):** Beinhaltet SuGaR. Für die Full-Scene-Baseline wird der unveränderte STS-Checkpoint verwendet; für die Objektrekonstruktion wird eine private Kopie der gefilterten Punktwolke mit maskenkonsistenter SuGaR-Optimierung verarbeitet. Danach folgen Poisson-Mesh-Extraktion, maskiertes UV-Textur-Baking und ein Multi-View-Konsens-Crop. Wegen tiefer Abhängigkeiten zum `diff-gaussian-rasterization` Modul wird hier strikt CUDA 11.8 erzwungen. Das intern geforderte Conda-Environment wird systemweit über eine Anpassung der `ENV PATH` Variable im Dockerfile nativ zugänglich gemacht.
- **Container E (Post-Processing):** Ein reiner **CPU-Container** (ohne GPU/CUDA-Abhängigkeit). Beinhaltet die C++ Bibliothek DGtal sowie die GDAL-Tools (`ogr2ogr`). Durch die Auslagerung dieser CPU-basierten Nachbearbeitung in einen eigenen Microservice bleibt die Architektur extrem leichtgewichtig, da keine massiven Nvidia-Images geladen werden müssen.

5 Risikomanagement und Datenhandling

Mehrere kritische Punkte und prozessuale Hürden erfordern besondere Aufmerksamkeit:

1. Photogrammetrischer Aufnahmestandard zur Qualitätssicherung und Vermeidung von Sub-Modellen:

Ein zentraler Erfolgsfaktor der Rekonstruktion linearer Bauteile ist die Akquisition des primären Fotomaterials. Um eine fehlerhafte Zerteilung der Punktwolke durch COLMAP in unvollständige Sub-Modelle (Splits) gänzlich zu verhindern, müssen strenge photogrammetrische Standards eingehalten werden:

- **Verriegelung der inneren Orientierung (Konstante Kameraparameter):** Jede Änderung der Brennweite während der Aufnahme destabilisiert die Ausgleichsrechnung fundamental. Der Autofokus der Kamera bzw. Drohnensensorik muss verpflichtend deaktiviert sein (Einstellung auf manuellen Fokus unendlich bzw. feste hyperfokale Distanz). Ebenso ist eine automatische Belichtung (Auto-Exposure) und ein automatischer Weißabgleich zu unterbinden, um photometrische Fehlzuordnungen der SIFT-Deskriptoren zu vermeiden.
- **Geometrische Überdeckung und Bildwinkel:** Die Aufnahmen müssen eine Längsüberdeckung von mindestens 80 % und eine Querüberdeckung von mindestens 60 % aufweisen. Bei linearen Objekten müssen dichte Nadir-Aufnahmen (90° orthogonal nach unten) planmäßig durch leicht geneigte Oblique-Aufnahmen (15° bis 30° Neigungswinkel) ergänzt werden. Dies bricht die koplanare Entartung der Sichtstrahlen auf und ermöglicht dem Bündelausgleich eine saubere Rekonstruktion zylindrischer Geometrien.

- **Einbringung von Anker-Geometrien im Alphanumerischen Raum (Anchor Features):** Extrem glatte, einfarbige Objekte (wie das nackte Erdkabel oder Kunststoffflaschen) besitzen keine SIFT-merkmalsfähige Textur, wodurch der photogrammetrische Strahlenschnitt kollabiert. Zur physikalischen Konditionierung der Szene müssen kontrastreiche, hochfrequente Muster-Strukturen („Anker-Objekte“ wie gemusterte Markierungsmatten, beschriebene Zeitungsseiten oder künstliche codierte ArUco-Markerboards) im unmittelbaren Nahbereich um das Objekt ausgelegt werden. Diese Anker erzeugen tausende fehlerfreie Feature-Matches, an welchen COLMAP die Kameras stabil orientieren kann, während das eigentliche Zielobjekt später durch die semantische 2D-Segmentierung (SAM 3.1) sauber herausgelöst wird.
2. **Speichermanagement:** Drohnenvideos in 4K bei 60 FPS erzeugen massive Datenmengen. Das Konzept sieht vor, dass diese temporär im nativen Linux-Dateisystem liegen und nach der 6 FPS Extraktion gelöscht werden, um I/O-Engpässe unter WSL2 zu vermeiden.
 3. **SAM 3 Tracking-Stabilität (VRAM) und Modellkonsistenz:**
 In der praktischen Ausführung zeigte sich ein nichtlinear wachsender VRAM-Bedarf im `propagate_in_video`-Pfad (insb. während `F.interpolate` in der Output-Erzeugung). Dadurch trat ein `CUDA out of memory` nicht deterministisch auf (z. B. um Frame 45, bei anderen Läufen früher oder später), obwohl dieselbe Eingabe verwendet wurde. Ursache ist die Kombination aus langem Sequenzgedächtnis (Memory-Modul), fragmentiertem Allocator-Zustand und voller HD-Auflösung über viele Frames.
 Als technische Gegenmaßnahme wurde die Propagation auf feste Zeitfenster (Chunking) umgestellt: Jeder Lauf verarbeitet nur ein Segment mit `start_frame_index` und `max_frame_num_to_track`; die Masken werden unmittelbar pro Frame persistiert, statt alle Outputs bis zum Ende im RAM/VRAM vorzuhalten. Zusammen mit Offloading-Flags (Video/State auf CPU) stabilisiert dies die Pipeline auf 16-GB-Karten.
 Ein zweiter, davon getrennter Befund ist die **Gewichts-/Architektur-Konsistenz:** Trotz stabilem Lauf können aufgrund von `Missing keys/Unexpected keys` beim Laden des SAM-Checkpoints formal gültige, aber inhaltlich leere Masken entstehen. Daher wird als Quality-Gate vor Übergabe an STS verbindlich geprüft: (i) Anteil nichtleerer Masken, (ii) temporale Kontinuität über benachbarte Frames, (iii) Sichtprüfung an Stichproben. Nur bei erfüllten Kriterien werden Masken in Container C übernommen.
 4. **[GELÖST: SAM 3.1 Video-Tracking stabilisiert, Stand 06/2026]**
 Die anfängliche Instabilität des SAM 3.1 Video-Pfades konnte durch drei gezielte Maßnahmen gelöst werden, was eine unterbrechungsfreie Voll-Propagation über alle 100 Frames auf einer 16-GB-GPU ermöglichte: (i) **VRAM-Limitierung durch sequenzielles Vision-Language-Matching:** Die Aktivierung von `batched_grounding_batch_size = 1` (anstelle von standardmäßigem Batching nach Text-Prompts) reduzierte den VRAM-Verbrauch von über 12 GB auf unter 5 GB, wodurch jegliche `CUDA out of memory`-Fehler verhindert wurden. Zusätzlich wurde die PyTorch-Compilation via `TORCH_COMPILE_DISABLE=1` deaktiviert, um massive RAM/VRAM-Spitzen vorzubeugen.
 (ii) **Behebung des Shape-Assertions-Fehlers:** Der zuvor auftretende Fehler

`AssertionError: pos_pred_mask.shape[0] == 1` bei der Chunk-Verarbeitung wurde durch Rückkehr zum globalen Video-Inferenz-Modus unter Verwendung der Standard-Konditionierungsparameter (`max_cond_frames_in_attn` unlimitiert) gelöst. (iii) **Datensatz-Kompatibilität bei der Masken-Persistierung:** SAM 3.1 Multiplex liefert den Vorhersage-Stream nicht in Form geschachtelter Dictionaries wie reguläre SAM-Modelle, sondern im Multiplex-spezifischen Format, d.h. als Dictionary mit den Hauptschlüsseln `out_obj_ids` und `out_binary_masks`. Ein angepasster Masken-Parser extrahiert nun die Binärmasken direkt über `out_binary_masks[idx]`. Durch diese Korrekturen konnte die Inferenz bei ca. 3,1it/s vollständig stabilisiert werden und liefert mit dem Text-Prompt "cable" reproduzierbar **33 nicht-leere Masken** über den 100 Frames großen Testdatensatz hinweg.

Einfache Erklärung (für Laien/Nicht-Techniker verständlich):

Zuvor brach das System beim Versuch, das Kabel im Video automatisch zu verfolgen, ständig ab oder fand schlichtweg gar nichts. Das lag an drei anschaulichen Problemen:

- *Die Speicher-Überlastung (VRAM-Absturz):* Das Programm wollte alle KI-Bildanalysen für die Text-Prompts gleichzeitig im Grafikspeicher lösen. Das ist so, als ob man versucht, 10 dicke Bücher gleichzeitig auf einmal aufzuschlagen und querzulesen – das Gehirn (die GPU) ist überfordert. Indem wir dem System befehlen, die Sätze brav nacheinander zu verarbeiten (**Batch-Größe = 1**) und das zeitaufwendige „Vorab-Auswendiglernen“ ausschalten (**Kompilierung aus**), liest die GPU das Video nun stressfrei Bild für Bild. Der Speicherbedarf sank von über 12 GB auf gemütliche 4,5 GB.
- *Die Denkblockade (Der AssertionError):* Um Speicher zu sparen, hatten wir das Video anfangs in kurze 8-Bild-Häppchen zerlegt. Die KI kam bei der Übergabe der Häppchen jedoch völlig durcheinander, verlor den Faden und behauptete stur, die Teile würden logisch nicht mehr zusammenpassen. Wir lassen das Video nun einfach elegant am Stück durchlaufen und überlassen der KI die vollautomatische Verwaltung ihres Langzeitgedächtnisses.
- *Das Übersetzungs-Missverständnis (Leere Maskenergebnisse):* Die KI hat das Kabel im Video zwar erfolgreich gefunden und markiert, dies aber in einer neuen „fremden Sprache“ (einem veränderten Ausgabeformat) dokumentiert. Unser Empfänger-Skript suchte die Ergebnisse beharrlich am alten Ablageort und meldete fälschlicherweise „nichts gefunden“. Der neu geschriebene Parser übersetzt diese neue Struktur nun fehlerfrei, wodurch die erzeugten Kabelmasken exakt erfassungskonform abgespeichert werden können.

5. **[GELÖST: SAM 3.1 lauffähig nach Laufzeit-Modernisierung, Stand 06/2026]**

Die zuvor beschriebenen Lade- und Stabilitätsprobleme ließen sich auf eine **nicht explizit fixierte, zu niedrige Python-Laufzeit** (vormals Python 3.10) zurückführen.

Da SAM 3.1 moderne Laufzeitkomponenten voraussetzt, kam es ohne explizite Versionsfestlegung wiederholt zu Import- und Kompatibilitätsfehlern, sodass die Umgebung erst nach mehreren Korrekturen wieder vollständig lief. Die Lösung war eine konsequente **Pinnung der gesamten Laufzeit** in Container A: `nvidia/cuda:12.6.3-cudnn` mit **Python 3.12, PyTorch 2.7.0+cu126** und **torchvision 0.22.0**.

Drei konkrete Stolpersteine wurden dabei behoben: (i) Unter Ubuntu 24.04 verhindert

die *externally-managed-environment*-Richtlinie (PEP 668) eine direkte `pip`-Installation; dies erfordert das Flag `--break-system-packages`. (ii) Beim Import des SAM-3.1-Pakets fehlten Laufzeitabhängigkeiten (`psutil`, `timm`), die explizit nachinstalliert werden mussten. (iii) Der Image-Pfad lädt standardmäßig das SAM-3-Gewicht; für SAM 3.1 muss der Checkpoint über `download_ckpt_from_hf(version="sam3.1")` ausdrücklich angefordert werden.

Ergänzend wurde ein **Einzelbild-Pfad** implementiert: Das Vorverarbeitungsskript erkennt nun automatisch, ob in `01_raw` ein Standbild (`.jpg/.jpeg/.png`) oder ein Video (`.mp4/.mov`) vorliegt, und nutzt für Bilder den text-geprompteten `build_sam3_image_model` with `Sam3Processor`. Dieser Pfad konnte auf einer statischen Testaufnahme mit dem Prompt “cable” reproduzierbar eine nichtleere Maske erzeugen. Der beim Laden weiterhin protokollierte `missing_keys`-Hinweis (vier `conv3`-Einträge des Vision-Backbones) blieb dabei folgenlos und steht der Maskengenerierung nicht entgegen.

6. [\[GELÖST: Punktwolken-Kollaps durch COLMAP Multi-Model-Splits behoben, Stand 06/2026\]](#)

Bei unruhigen Kameraführungen oder homogenen Texturen bricht die geometrische Feature-Matching-Stufe von COLMAP die Rekonstruktion häufig in disjunkte Teilmodelle auf, welche sequenziell unter `sparse/0`, `sparse/1` usw. abgelegt werden. Im konkreten Testszenario lag dies vor allem an der ***extrem kontrastarmen, homogenen Umgebung*** (das blaue Kabel lag auf einem weißen, nahezu texturlosen Tisch). Die SIFT-Feature-Detektion konnte dadurch auf der Tischoberfläche kaum homologe Bildpunkte extrahieren, was zu einem häufigen Abreißen der zeitlichen Bildüberdeckung führte.

Das Dataloading-Skript von Segment-then-Splat (`prep_sts_scene.py`) war starr darauf konfiguriert, stets aus `sparse/0` zu lesen. Bei unvollständigen Initialisierungen enthielt dieses Modell jedoch oft nur minimale Artefakte (z. B. 2 registrierte Kameras mit gerade mal 1 spärlichen Raumpunkt auf dem Kabel), was im nachfolgenden 3DGS-Optimierer zum totalen geometrischen Kollaps des Trassenbereichs führte.

Die Behebung dieses Hinderungsauftritts erfolgt auf zwei komplementären Ebenen:

- (a) **Dynamischer, dateigrößenbasierter Best-Model-Resolver (Teilmodell-Wechsler):** In `prep_sts_scene.py` scannt eine Suchfunktion vor dem STS-Training sämtliche von COLMAP generierten Unterordner und ermittelt über die Dateigröße der binären Punktwolkendatenbank (`points3D.bin`) vollautomatisch die dichteste und vollständigste Hauptrekonstruktion (z. B. `sparse/2` mit 123 Bildern und 89 validen Stützpunkten auf dem Kabel). Das Skript kopiert diese ausgewählte Wolke dynamisch als Haupt-Initialisierungs-Skelett nach `/data/05_3dgs/sparse/0`. Hierdurch konnte die nutzbare Kabelpunktwolke von 1 auf 89 Punkte vervielfacht werden.
- (b) **Stabilisierung der SfM-Parametersätze in Container B (`run_sfm.sh`):** Um das Aufspalten in Teilmodelle präventiv direkt beim Feature-Matching zu unterdrücken, wurden die COLMAP-Parameter für kontrastarme Szenen angepasst. Bei der Merkmalsextraktion wird die Empfindlichkeit über `--SiftExtraction.ppe` 0.003 verschärft und das Limit auf 16384 Features erhöht, um schwache Kontraste auf der Tischplatte zu erfassen. Beim Matching reaktiviert `--FeatureMatching.guided_matching` 1 über die Epipolargeometrie zusätzliche Paare. Der Mapper selbst toleriert nun

über `--Mapper.abs_pose_min_num_inliers 15` und `--Mapper.min_num_matches 10` registrierte Kameras mit geringerer Punktdichte im selben Modell.

Einfache Erklärung (für Laien/Nicht-Techniker verständlich):

Bei der 3D-Rekonstruktion versucht das Programm COLMAP, hunderte Fotos wie ein Puzzle passend aneinanderzureihen. Da das blaue Kabel auf einem extrem gleißenden, weißen Tisch lag, gab es fast keine sichtbaren Details auf der Tischplatte. Für den Computer sieht die weiße Fläche aus jedem Winkel absolut identisch aus. Das ist so, als ob man versucht, ein Puzzle zu lösen, das nur aus blauem Himmel besteht – man verliert sofort den Faden. Statt dem *einen* großen Puzzle baut COLMAP heimlich mehrere kleine, voneinander getrennte Puzzle-Häufchen (Teilmodelle) auf. Unser nachfolgendes Trainingsprogramm greift standardmäßig blind in das erste Körbchen (Modell 0). Blöderweise lag in diesem Körbchen diesmal nur ein winziges Randfragment, auf dem sich gerade mal ein einziger Punkt unseres Kabels befand – das eigentliche Kabel lag vollständig in Körbchen Nummer 2! Da das System somit nur 1 Kabel-Punkt zum Lernen hatte, löste sich das Kabel im 3D-Modell in Luft auf.

Durch unsere doppelte Neuerung prüft ein intelligenter Suchgurt nun vorab die Größe aller Körbchen, identifiziert das vollwertige Hauptpuzzle (Modell 2 mit 89 Kabelpunkten) und übergibt dieses an das Training. Gleichzeitig haben wir der COLMAP-Kamera eine „Lupe“ aufgesetzt: Sie sucht nun viermal so detailreich nach winzigsten Staubkörnchen oder Mustern auf dem weißen Tisch, bemüht sich noch intensiver, Bildpaare wiederzuerkennen, und verlangt weniger perfekte Puzzleteile, um ein Bild an das Hauptmodell anzuhängen. Dadurch bleibt die Rekonstruktion zusammenhängend und das Kabel wird stabil lückenlos sichtbar.

7. **Das Georeferenzierungs-Paradoxon:** Da die objektspezifische Selektion (STS) nur das Kabel-Mesh isoliert, verschwinden im finalen 3D-Modell auch die am Boden ausgelegten Passpunkte (GCPs). Eine Post-Training Transformation des Modells durch manuelles Anklicken ist somit nicht anwendbar. Die Lösung ist eine *Pre-Training Georeferenzierung*: Die Open-Source-Software **CloudCompare** bietet hierfür ein ideales GUI. Da CloudCompare intern mit 32-Bit Single-Precision-Gleitkommazahlen arbeitet, führt eine direkte Eingabe von großen globalen UTM-Koordinaten ($> 10^5$) zu dramatischem Präzisionsverlust (1–10 cm). Daher wird eine *relative Georeferenzierung* durchgeführt: Ein ausgewählter GCP dient als lokaler Ursprung (Ankerpunkt) mit den Koordinaten (0, 0, 0), und alle anderen GCPs werden relativ zu diesem berechnet ($GCP_{\text{relativ}} = GCP_{\text{UTM}} - GCP_{\text{Anker}}$). Man lädt die unmaskierte, spärliche SfM-Punktvolke aus COLMAP in CloudCompare, ordnet die Passpunkte den relativen GCP-Koordinaten manuell zu (*Point Picking*) und berechnet eine 4x4-Transformationsmatrix (Kombination aus 3x3-Rotation, Translation und optionaler Skalierung). SuGaR berechnet danach parallel das rohe, lokale 3D-Mesh. Anstatt jedoch das rechenintensive Millionen-Polygon-Mesh in globale Koordinaten zu überführen, wird zunächst DGtal auf dem lokalen Mesh ausgeführt. Die resultierende lokale Centerline (bestehend aus wenigen hundert Punkten) wird in Container E über ein kurzes Python-Hilfsskript mittels der 4x4-Transformationsmatrix multipliziert, um die Rotation und Skalierung anzuwenden, und danach durch Addition des Anker-GCPs in die globalen UTM-Koordinaten überführt.

8. [NEUE ERKENNTNIS: Datenformate & GIS-Export]

ArcGIS Pro unterstützt .ply-Geometrien nicht nativ als Feature-Layer. Zudem verursacht das .obj-Format bei großen UTM-Koordinaten signifikante Präzisionsverluste (*Vertex Jittering*), da es standardmäßig als *Single Precision* verarbeitet wird. Die Pipeline splittet den Datenexport daher gezielt in zwei Endprodukte auf:

- **Das analytische Endprodukt (Centerline):** Da DGtal (als diskrete Geometrie-Bibliothek) im lokalen Raum um den Ursprung (0,0,0) am präzisesten und speichereffizientesten arbeitet, wird die Centerline zuerst auf dem lokalen .ply-Mesh berechnet. Ein Python-Hilfsskript in Container E wendet die 4x4-Transformationsmatrix an, addiert den globalen Ankerpunkt und gibt eine georeferenzierte CSV-Datei aus. Die Konvertierung dieser CSV-Datei in standardisierte GIS-Formate (wie GeoJSON) erfolgt danach Open-Source über den **GDAL/ogr2ogr Post-Processing Container**.
- **Das visuelle Endprodukt (BIM-Mesh):** Falls TenneT das Kabel zusätzlich als 3D-Objekt visualisieren möchte, wird das lokale .obj-Mesh vorab (in CloudCompare oder über ein Python-Skript) mit dem Rotationsteil der 4x4-Matrix rotiert und skaliert. Anschließend wird die geänderte Geometrie in ArcGIS Pro über das nativ integrierte Geoprocessing-Tool "Import 3D Files" eingelesen und der Einfügepunkt (Insertion Point / Global Shift) direkt auf die UTM-Koordinaten des Anker-GCPs gesetzt, was den Präzisionsverlust des .obj-Formats elegant umgeht.

6 Wissenschaftliche Evaluationsmetriken

Für die wissenschaftliche und ingenieurstechnische Validierung der Pipeline werden zwei Kategorien von Metriken eingesetzt: geometrische Metriken für die Lagegenauigkeit des Endprodukts und photometrische Metriken für die visuelle Rekonstruktionsqualität des 3DGS-Modells.

6.1 Geometrische Metriken (Lage- und Höhengenaugigkeit)

Um nachzuweisen, dass die extrahierte Centerline die geforderte Toleranz von ± 10 cm einhält, werden die Abweichungen zu den hochpräzisen GNSS-Referenzmessungen quantifiziert.

- **B-Spline-Referenzkurve:** Die GNSS-Referenzmessungen liegen als diskrete, dreidimensionale Koordinatenpunkte $P_i = (x_i, y_i, z_i)$ vor. Ein direkter Punkt-zu-Punkt-Vergleich mit der dichter diskretisierten DGtal-Centerline ist fehleranfällig, da die Punkte nicht exakt korrespondieren. Daher wird eine kontinuierliche dreidimensionale **Basis-Spline-Kurve (B-Spline)** $S(t)$ durch die GNSS-Punkte interpoliert bzw. angenähert. Der B-Spline garantiert eine mathematisch glatte (zweifach stetig differenzierbare, C^2 -konsistente) Referenzachse des realen Kabels.
- **RMSE (Root Mean Square Error):** Der RMSE berechnet den quadratischen Mittelwert der euklidischen Abstände zwischen den Punkten C_j der extrahierten

DGtal-Centerline und ihren jeweils orthogonalen Projektionen auf die B-Spline-Referenzkurve $S(t)$:

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{j=1}^N \min_t \|C_j - S(t)\|^2}$$

Der RMSE gibt Aufschluss über die durchschnittliche systematische und stochastische Abweichung über den gesamten Verlauf der Trasse.

- **Hausdorff-Distanz:** Während der RMSE die mittlere Abweichung beschreibt, identifiziert die **Hausdorff-Distanz** d_H den worst-case Fehler (maximale lokale Abweichung). Sie ist definiert als das Maximum der minimalen Abstände zwischen der extrahierten Kurve C und der Referenzkurve S :

$$d_H(C, S) = \max \left\{ \sup_{c \in C} \inf_{s \in S} \|c - s\|, \sup_{s \in S} \inf_{c \in C} \|c - s\| \right\}$$

Für den Praxisbezug (TenneT) ist die Hausdorff-Distanz die entscheidende Metrik: Nur wenn $d_H(C, S) \leq 10$ cm gilt, ist garantiert, dass das Kabel an keiner einzigen Stelle der Trasse die ingenieurstechnische Toleranzgrenze überschreitet.

6.2 Photometrische Metriken (Rekonstruktionsqualität)

Diese Metriken messen ausschließlich die optische Qualität der 3D-Rekonstruktion. Sie werden **während des Trainings von Segment-then-Splat (STS)** auf gehaltenen Testbildern (Novel View Synthesis) berechnet, um sicherzustellen, dass die Geometrie der Szene visuell korrekt und ohne Artefakte erfasst wurde. Sie haben jedoch keinen direkten Bezug zur finalen UTM-Georeferenzierung.

- **PSNR (Peak Signal-to-Noise Ratio):** Misst das Verhältnis zwischen dem maximal möglichen Signalwert und dem störenden Rauschen im gerenderten Bild im Vergleich zum Ground-Truth-Foto. Der Wert wird logarithmisch in Dezibel (dB) angegeben. Höhere Werte (typischerweise > 28 dB im 3DGS) deuten auf eine präzise Rekonstruktion hin, wobei PSNR sehr empfindlich auf pixelgenaue Verschiebungen reagiert.
- **SSIM (Structural Similarity Index):** Bewertet im Gegensatz zu PSNR nicht nur den Pixelabstand, sondern die strukturelle Ähnlichkeit, Helligkeit und den Kontrast in lokalen Bildbereichen. SSIM liegt zwischen 0 und 1 (wobei 1 eine perfekte strukturelle Übereinstimmung beschreibt) und entspricht besser der menschlichen visuellen Wahrnehmung.
- **LPIPS (Learned Perceptual Image Patch Similarity):** Nutzt ein tiefes neuronales Netzwerk (z.B. VGG), um die perzeptuelle Ähnlichkeit auf Feature-Ebene zu vergleichen. Ein niedrigerer LPIPS-Wert zeigt an, dass das Rendering für das menschliche Auge (insbesondere bezüglich feiner Texturen und Schärfe) dem Originalfoto gleicht. Dies ist entscheidend, um zu prüfen, ob dünne Linienstrukturen verschwommen sind.

7 Zeit- und Scope-Planung

Projektarbeit (PA) – Fokus Machbarkeit (4 Wochen):

- Konfiguration der Docker-Architektur (Container A, B, C, D & E).
- Master-Skript Orchestrierung inkl. manuellem Breakpoint (CloudCompare).
- **Georeferenzierungs-Pipeline:** Implementierung der Transformationslogik (Berechnung der 4x4-Ähnlichkeitstransformationsmatrix in CloudCompare und automatisierte Transformation der extrahierten Centerline in Container E).
- Nachweis der Funktionalität der Segment-then-Splat Pipeline (Objekt-Loss) und Meshing (SuGaR) am Testdatensatz (Aluabspernung).
- *Optional:* Erste Voruntersuchungen zu SAM 2/3 und SAHI Tiling.

Bachelorarbeit (BA) – Fokus Skalierung & Metrik (10 Wochen):

- **Feldarbeit:** Drohnenflug sowie parallele GNSS-Referenzmessung der ausgelegten GCPs und der Kabel-Scheitelachse (*Centerline*) an der realen Trasse.
- Centerline-Extraktion via DGtal auf dem realen Kabel-Sub-Mesh.
- **Wissenschaftliche Evaluation (Metriken):** Berechnung des Root Mean Square Error (**RMSE**) und der maximalen **Hausdorff-Distanz** der extrahierten 3DGS-Centerline gegenüber einer interpolierten **Basis-Spline-Kurve (B-Spline)** aus den diskreten GNSS-Referenzmessungen, um die Einhaltung der ± 10 cm Toleranz nachzuweisen.
- Wirtschaftlichkeitsanalyse für TenneT.

Details der Parameterstudie und Sensitivitätsanalyse (BA)

Um die Robustheit und Präzision der entwickelten Pipeline unter realen Bedingungen nachzuweisen, wird im Rahmen der Bachelorarbeit eine systematische Parameterstudie durchgeführt. Dabei werden folgende Einflussfaktoren evaluiert:

- **GCP-Konfiguration (Anzahl und Verteilung):** Vergleich einer minimalen Konfiguration (3 GCPs) mit erweiterten Setups (5 bis 7 GCPs). Zudem wird die geometrische Anordnung untersucht: Lineare Platzierung (entlang des Kabelgrabens) versus stufenförmige/Zickzack-Verteilung (zur Stabilisierung der Rotationsachse um die Trassenachse).
- **Aufnahmeparameter (Flughöhe und Neigungswinkel):** Evaluierung des Einflusses der Flughöhe (5 m vs. 10 m vs. 15 m) auf die Ground Sampling Distance (GSD) und die Segmentierungsqualität. Vergleich von Nadir-Aufnahmen (90°) mit Oblique-Aufnahmen (15°–30° Neigung), um die zylindrische Geometrie der Kabelunterkante rauscharm zu rekonstruieren.
- **Segmentierungs- und Tracking-Modelle (Frontend-Vergleich):** Systematischer Vergleich zwischen dem SAM 3 Video Predictor, SAM 2 und SAHI Tiling (Slicing). Untersuchung des Einflusses der Frame-Extraktionsrate (z. B. jeder 5. vs. 10. vs. 20. Frame) auf die Rechengeschwindigkeit und die temporale Maskenkonsistenz zur Vermeidung von 3D-Rekonstruktionsfehlern.
- **Rekonstruktions- und Meshingparameter (STS & SuGaR):** Parametertuning der Poisson-Rekonstruktionstiefe (*Poisson Depth* 7, 8, 9 und 10) in SuGaR, um den

optimalen Kompromiss zwischen der Erkennung des realen Kabeldurchmessers und der Rauschunterdrückung zu finden.

- **Kurvenglättung (DGtal-Postprocessing):** Vergleich verschiedener Glättungsalgorithmen (z. B. B-Spline-Approximation oder gleitender Mittelwert) zur Rauschfilterung der extrahierten 3D-Kurve.

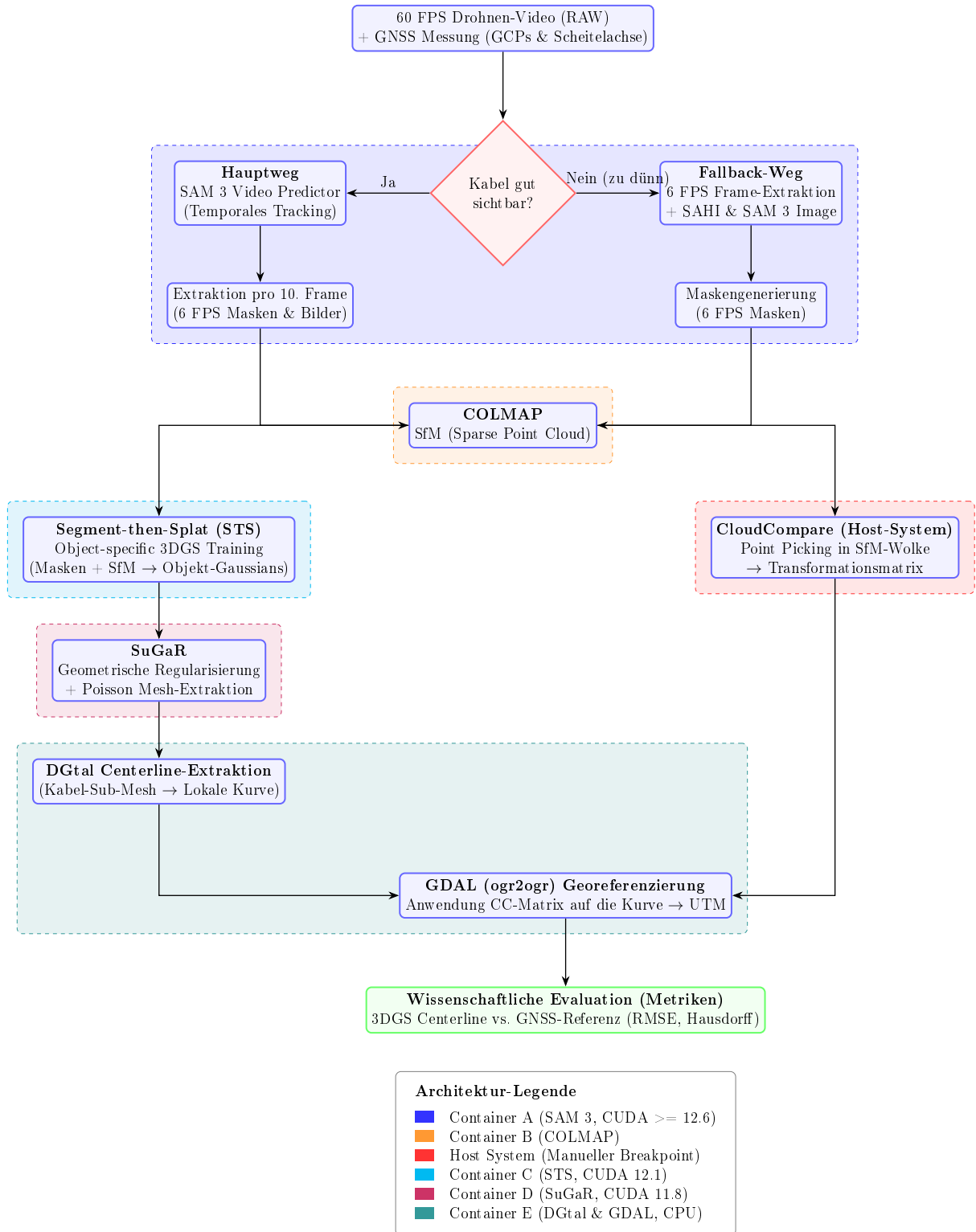


Abbildung 1: Microservice-Architektur (v4): 5-Container-Pipeline mit Segment-then-Splat für objektspezifisches Training und SuGaR für geometrisches Meshing.

8 Anhang: Single Source of Truth (SSOT) – Pipeline Installation

Dieses Dokument dient als zentrale Wahrheit (SSOT) für die Installation und Orchestrierung der gesamten 5-Container-Pipeline. Es fasst alle Hard- und Software-Abhängigkeiten zusammen, um Reproduzierbarkeit zu gewährleisten.

Hardware-Grundvoraussetzungen

- **GPU:** CUDA-kompatible Nvidia GPU (Compute Capability 7.0+).
- **VRAM:** 24 GB (erforderlich, um in Paper-Evaluations-Qualität zu trainieren).
- **OS:** Windows 10/11 mit WSL2 oder natives Ubuntu Linux 22.04.

Container A: SAM 3 (Segmentierung)

Zuständig für die 2D-Maskierung des Drohnenvideos. Dieser Container nutzt modernste Meta-Abhängigkeiten.

Requirements (laut offizieller Vorgabe):

- Python 3.10 or higher
- PyTorch 2.4 or higher (im Setup wird PyTorch mit der passenden CUDA-Laufzeitumgebung installiert)
- CUDA-kompatible GPU mit CUDA $\geq$ 12.6

Implementierungsentscheidung: Docker vs. Conda

Für die Ausführung der SAM 3.1 Codebasis wurde konzeptionell von einer reinen lokalen Conda-Umgebung Abstand genommen. Stattdessen läuft SAM 3.1 vollständig in einem dedizierten Docker-Container (`sam3-preprocess`).

Gründe gegen Conda und für Docker:

1. **CUDA-Konflikte:** SAM 3.1 verlangt moderne PyTorch-Versionen (z. B. 2.7.0 mit CUDA 12.6) und greift auf experimentelle C++ Extensions (`sam3_cu`) zurück. In einer lokalen Anaconda-Umgebung kommt es hier mit den bestehenden CUDA 11.8 Treibern von COLMAP/SuGaR ständig zu "Layer Mismatches" und Kompilierungsfehlern. Docker kapselt die CUDA 12.6 Abhängigkeiten sauber vom Host-System ab.
2. **Abhängigkeits-Knoten (Dependency Hell):** Das SAM 3.1 Release benötigt tiefgreifende Bibliotheken (`pycocotools`, `einops`, `timm`), die nativ im Container über `apt-get` und `pip` verlässlich zusammengestellt werden.
3. **Hardware-Starvation (Host-Safety):** Das Laden von SAM 3.1 in Full-HD (`sam3.1_multiplex`) absorbiert immense RAM- und CPU-Ressourcen beim Instanzieren des Video-Predictors. Im Container können via `docker-compose.yml` die CPU-Berechnungen strikt gedrosselt werden (`OMP_NUM_THREADS=8`), um Systemabstürze effektiv zu verhindern. Ein VRAM- und RAM-Offloading ist zusätzlich im inferierenden Python-Skript konfiguriert.
4. **Hugging Face Token-Sicherheit:** Über Docker-Secrets (`.env` / `HF_TOKEN`) wird die Authentifizierung für Gated Models sicher gehandhabt, ohne lokale Systemumgebungen zu belasten.

Installationsablauf (Docker):

Die Bereitstellung erfolgt vollautomatisch über das dedizierte `Dockerfile` auf Basis von `pytorch/pytorch:2.7.0-cuda12.6-cudnn9-devel`. Dies klonet das offizielle Repository, baut die C++ Extensions und cacht über das Hugging Face Hub das 3.5 GB schwere Multiplex-Gewicht persistent ins Host-Volume.

Ausführung & Masken-Export (Python API):

Hier ist das Python-Skript zur temporalen Propagation des Kabel-Prompts im Video. Da der SAM 3 Video Predictor im Tracking-Modus aus Konsistenzgründen nur eine einzelne, eindeutige Maske pro Frame propagiert (und nicht die drei hierarchischen Skalen des Image Predictors), generiert das Skript die für das Curriculum-Learning von Segment-then-Splat (STS) benötigten Stufen (`small`, `middle`, `default`) künstlich über morphologische Bildoperationen (Erosion und Dilation):

```
import os
import cv2
import numpy as np
from PIL import Image
from sam3.model_builder import build_sam3_video_predictor

# 1. Video Predictor auf GPU 0 initialisieren
predictor = build_sam3_video_predictor(gpus_to_use=[0])

# 2. Session mit dem Drohnen-Video starten
video_path = "/data/video.mp4"
response = predictor.handle_request(
    request=dict(type="start_session", resource_path=video_path)
)
session_id = response["session_id"]

# 3. Prompt setzen (z. B. Punkt-Prompt auf das Kabel in Frame 0)
predictor.handle_request(
    request=dict(
        type="add_prompt",
        session_id=session_id,
        frame_index=0,
        points=[[1024, 768]], # Beispielkoordinaten auf dem Kabel
        labels=[1]
    )
)

# 4. Stream-Propagation & Multi-Scale Export via Morphologie
output_dir = "/data/masks"
os.makedirs(output_dir, exist_ok=True)

# Definiere Kernel für Erosion und Dilation (Größe auf Kabeldurchmesser anpassen)
kernel = np.ones((5, 5), np.uint8)
```

```

# propagate_in_video liefert einen Generator
for response in predictor.handle_stream_request(
    request=dict(type="propagate_in_video", session_id=session_id)
):
    frame_idx = response["frame_index"]
    outputs = response["outputs"]
    obj_ids = outputs["out_obj_ids"]
    binary_masks = outputs["out_binary_masks"] # Shape: [num_objects, H, W]

    # Erzeuge einen Unterordner pro Frame-Index
    frame_dir = os.path.join(output_dir, f"frame_{frame_idx:05d}")
    os.makedirs(frame_dir, exist_ok=True)

    for idx, obj_id in enumerate(obj_ids):
        mask = binary_masks[idx] # Boolesches Array [H, W]
        if mask.any():
            # Konvertierung: True -> 255, False -> 0
            mask_uint8 = (mask * 255).astype(np.uint8)

            # 1. Middle (Original)
            img_middle = Image.fromarray(mask_uint8)
            img_middle.save(os.path.join(frame_dir, "middle.png"))

            # 2. Small (Erodiert für den stabilen Kabelkern)
            mask_small = cv2.erode(mask_uint8, kernel, iterations=1)
            img_small = Image.fromarray(mask_small)
            img_small.save(os.path.join(frame_dir, "small.png"))

            # 3. Default (Dilatiert für den Sicherheitsrand)
            mask_default = cv2.dilate(mask_uint8, kernel, iterations=1)
            img_default = Image.fromarray(mask_default)
            img_default.save(os.path.join(frame_dir, "default.png"))

# 5. Session sauber schließen
predictor.handle_request(
    request=dict(type="close_session", session_id=session_id)
)

```

Container B: COLMAP (Structure from Motion)

Generiert die Kameraposen und die Sparse Point Cloud aus den maskierten Drohnenbildern. Aufgrund der massiven Architekturänderungen (Einführung von GLOMAP) in Release 4.0.0, wird explizit zur stabileren Patch-Version **4.0.4** geraten, welche kritische Bugs (Speicherlecks, Race-Conditions in der Datenbank) behebt.

Quellen & Docker-Image:

- **Releases:** <https://github.com/colmap/colmap/releases>

- **Docker-Image:** `docker pull colmap/colmap:4.0.4-cuda`

Ausführung & PLY-Export (für CloudCompare):

Da COLMAP die Punktwolke standardmäßig in einem proprietären Binärformat (.bin) speichert, welches von CloudCompare nicht direkt gelesen werden kann, wird die Punktwolke direkt im Anschluss an die Rekonstruktion über den `model_converter` in eine standardisierte .ply-Datei exportiert. Um eine geometrische Verformung der Punktwolke (ebene Entartung/Kollaps) während der Rekonstruktion unkalibrierter Drohnenbilder zu verhindern, wird die Parametrisierung basierend auf modernsten Photogrammetrie-Workflows angepasst (SIMPLE_RADIAL Kameramodell, erzwungenes gemeinsames Objektiv via `single_camera 1` und sequentielles Matching).

1. Feature-Extraktion (Kameramodell einschränken & gleiches Objektiv forcieren)

```
colmap feature_extractor \
  --database_path /data/scene/database.db \
  --image_path /data/scene/images \
  --ImageReader.camera_model SIMPLE_RADIAL \
  --ImageReader.single_camera 1
```

2. Keypoint-Matching (Sequentiell mit Overlap statt Exhaustive für Videoüberhang)

```
colmap sequential_matcher \
  --database_path /data/scene/database.db \
  --SequentialMatching.overlap 15
```

3. Structure-from-Motion (Sparse Reconstruction)

```
mkdir -p /data/scene/sparse
colmap mapper \
  --database_path /data/scene/database.db \
  --image_path /data/scene/images \
  --output_path /data/scene/sparse
```

4. Export der Punktwolke als PLY-Format

```
colmap model_converter \
  --input_path /data/scene/sparse/0 \
  --output_path /data/scene/sparse/points3D.ply \
  --output_type PLY
```

Hintergrund zur mathematischen Optimierung (für „Dummis“ erklärt):

Wenn wir ein Drohnenvideo Frame-für-Frame in COLMAP einspeisen, stehen wir vor zwei massiven mathematischen Hürden:

1. **Die Overfitting-Falle (Zu viele Freiheitsgrade):** Wenn wir COLMAP ein extrem komplexes Kameramodell (wie OPENCV) geben, versucht der Algorithmus, für jedes der hunderte Bilder Brennweiten und Objektivkrümmungen völlig unabhängig voneinander zu schätzen. Das führt zu einer geometrischen Katastrophe: COLMAP „schummelt“ mathematisch, verbiegt die Parameter extrem und presst die gesamte 3D-Welt flach wie eine Flunder auf eine einzige Ebene zusammen, um Restfehler zu minimieren. Mit SIMPLE_RADIAL verbieten wir diese Schummelei, indem wir nur noch vier physikalische Kern-Parameter zulassen. Gleichzeitig zentrieren wir über `-ImageReader.single_camera 1` alle Rechnungen auf genau *ein* einziges, geteiltes Objektiv.

2. **Die Basislinien-Falle (Fast-parallele Sehstrahlen):** Wenn zwei aufeinanderfolgende Bilder nur Millisekunden auseinander liegen, bewegt sich die Drohne kaum fort. Die Sehstrahlen der Kameras schneiden sich unter einem extrem spitzen Winkel (0°). COLMAP kann die Tiefe dadurch nicht verlässlich berechnen und flacht die Punktwolke ab. Indem wir über `SAM3_FRAME_STEP` künstlich nur jedes z. B. 5. oder 10. Bild rekonstruieren, vergrößern wir den physischen Abstand (die Basislinie) zwischen den Kamera-Standorten. Dadurch schneiden sich die Sehstrahlen in einem gesunden, steilen Dreieck und zaubern eine plastische, tiefe, dreidimensionale 3D-Struktur hervor.
3. **Die Treffsicherheits-Falle (Falsche Übereinstimmungen):** Das standardmäßige `exhaustive_matcher` vergleicht jedes Bild mit absolut jedem anderen Bild. Bei sich wiederholenden Texturen (z. B. Gleisen, Rohren oder Asphaltmustern) führt das zu fatalen Fehlverknüpfungen (False Loop Closures). Indem wir auf den `sequential_matcher` mit einem Overlap von 15 Frames umschalten, vergleicht COLMAP Bilder nur noch mit ihren unmittelbaren zeitlichen Nachbarn (im Video naheliegend). Das ist nicht nur hunderte Male schneller, sondern schützt die 3D-Punktwolke hochgradig effektiv vor geometrischer Verwerfung.
4. **Die CLI-Versions-Falle (Veraltete Befehle):** Beim Upgrade auf modernere, offizielle Docker-Images (wie `colmap/colmap:latest` ab Version 4.0) scheitern klassische, ältere Skripte an deprecated Parametern. Parameter wie `-SiftExtraction.use_gpu` oder `-SiftExtraction.max_image_size` wurden aus dem CLI-Befehlssatz gestrichen, da die moderne CUDA-Hardware-Beschleunigung und das Auto-Resizing heute vollautomatisch und optimal aufeinander abgestimmt nativ im Kernel von COLMAP implementiert sind. Das Entfernen dieser Parameter garantiert die reibungslose Ausführung auf modernen GPGPU-Knoten.
5. **Die CHOLMOD-Numerik-Falle und Levenberg-Marquardt-Instabilität:** Während der Bildregistrierung im `colmap mapper` taucht wiederholt die Warnmeldung auf:
`Linear solver failure. Failed to compute a step: CHOLMOD warning: Matrix not positive definite.`
****Wissenschaftliche Erklärung:**** COLMAP nutzt für die Bündelblockausgleichung (*Bundle Adjustment* / gemeinsame Optimierung von 3D-Punkten und Kameraposen) standardmäßig den quadratischen Optimierungsalgorithmus nach ****Levenberg-Marquardt (LM)**** via *Ceres-Solver*. Um die Schrittweite Δx zu berechnen, löst Ceres in jeder Iteration das linearisierte Normalgleichungssystem:

$$(J^\top J + \mu I) \Delta x = -J^\top f$$

wobei J die Jacobi-Matrix der partiellen Ableitungen, μ der Dämpfungsparameter und f der Vektor der Reprojektionsfehler ist. Die Koeffizientenmatrix $H = J^\top J + \mu I$ ist theoretisch symmetrisch und positiv-definit (alle Eigenwerte > 0), was die Nutzung der sehr schnellen und numerisch stabilen ****Cholesky-Zerlegung**** (implementiert über die SuiteSparse-Bibliothek *CHOLMOD*) erlaubt.

In unserem Testszenario (weißes, kontrastarmes Tischtuch mit wenigen, verrauschten Punktkorrespondenzen) degeneriert das System jedoch geometrisch. Wenn künstlich verrauschte oder geometrisch kollineare Strahlen geschnitten werden sollen, verliert die Jacobi-Matrix ihren vollen Spaltenrang. Dadurch kollabiert die Matrix $J^\top J$

stellenweise zu einer singulären Matrix (Rangdefekt / unvollständiges Gleichungssystem mit unendlich vielen Scheinlösungen). Die Dämpfung μ ist temporär zu klein, um dies auszugleichen, wodurch die Matrix H numerisch nicht-positiv-definit wird (einige Eigenwerte schrumpfen auf ≤ 0).

Numerische Konsequenz: CHOLMOD bricht die Cholesky-Zerlegung $H = LL^\top$ an einer Division durch Null (oder Wurzel aus einer negativen Zahl) ab. Ceres fängt diese Warnung jedoch intern robust ab: Das System vergrößert im nächsten Schritt den Dämpfungskoeffizienten μ drastisch (Gefälle wechselt vom Newton-Schritt in Richtung des steilsten Abstiegs / Gradient Descent). Dadurch wird die Matrix wieder numerisch stabilisiert und der Schritt wiederholt. Die Warnung ist somit **ein klarer Indikator für extreme Texturarmut** der Szene, führt jedoch dank der internen Dämpfungsdynamik von Ceres **nicht** zum Absturz der gesamten Rekonstruktion, solange genügend gesunde Bildbereiche verbleiben.

Breakpoint: Nach diesem Schritt pausiert das Master-Skript automatisch. Die erzeugte Datei `points3D.ply` wird auf dem Host-System in CloudCompare geladen, um die manuelle Georeferenzierung via GCP-Point-Picking durchzuführen. Nach dem Speichern der Transformationsmatrix wird das Skript fortgesetzt.

Container C: Segment-then-Splat (Object-specific 3DGS)

Führt das objektspezifische 3DGS-Training durch. STS weist jedem initialen COLMAP-Gaussian eine Object-ID zu (basierend auf den SAM 3 Masken) und trainiert mit dem nativen Object-specific Loss ($\mathcal{L}_{obj}$). Das intern vorgesehene AutoSeg-SAM2 wird vollständig deaktiviert und durch die externen SAM 3 Masken aus Container A ersetzt. Hierfür wird der Dataloader (z. B. in `scene/dataset_readers.py` oder `scene/cameras.py`) so angepasst, dass er pro Frame die drei Masken-Dateien (`small.png`, `middle.png`, `default.png`) einliest und in das Dictionary `viewpoint_cam.object_masks` einspeist.

Requirements (Docker-Umgebung):

- **Base Image:** `nvidia/cuda:12.1.0-devel-ubuntu22.04`
- Python 3.10, PyTorch 2.3.1 mit CUDA 12.1
- 24 GB VRAM

Installationsablauf:

```
git clone https://github.com/luyr/Segment-then-Splat --recursive
cd Segment-then-Splat
```

```
conda create -n segment_then_splat python=3.10
conda activate segment_then_splat
pip install torch==2.3.1 torchvision==0.18.1 torchaudio==2.3.1 \
--index-url https://download.pytorch.org/whl/cu121
pip install -r requirements.txt --no-build-isolation
```

Anpassung des Dataloaders und Trainingsablauf: Im Datensatz-Reader wird die folgende Logik implementiert, um die Maskentensoren direkt zu laden:

```

# Pseudo-Code zur Anpassung des Kamera-Ladevorgangs:
import torch
from PIL import Image

def load_object_masks(frame_dir):
    scales = ["small", "middle", "default"]
    object_masks = {}
    for scale in scales:
        mask_path = os.path.join(frame_dir, f"{scale}.png")
        if os.path.exists(mask_path):
            # Graustufen laden, Normalisieren und als Boolean-Tensor konvertieren
            mask_img = Image.open(mask_path).convert("L")
            mask_np = np.array(mask_img) > 127
            mask_tensor = torch.from_numpy(mask_np).bool() # Shape: [H, W]
            object_masks[scale] = [mask_tensor]
        else:
            object_masks[scale] = []
    return object_masks

# Zuweisung zur viewpoint_cam während der Szenen-Initialisierung:
# viewpoint_cam.object_masks = load_object_masks(f"/data/masks/frame_{frame_idx:05d}")

```

Ausführung des Trainings:

```

# 1. Object-specific Initialization (Zuweisung der initialen SfM-Punkte)
# STS prüft anhand der default/middle/small Masken die Punkt-Sichtbarkeiten
python ./helpers/object_specific_initialization.py \
    --scene_root /data/scene/

# 2. Training mit dem geänderten Dataloader und Object-specific Loss
python train.py -s /data/scene/ -m /data/output \
    --eval --iterations 40000 \
    --num_sample_objects 3 \
    --densify_until_iter 20000 \
    --partial_mask_iou 0.3

```

Wichtiger Hinweis: Durch die Entkopplung der Maskenberechnung läuft die CUDA-spezifische Optimization in Container C völlig autark von den CUDA-Anforderungen des SAM 3 Frontends in Container A.

Container D: SuGaR (Geometrische Regularisierung & Meshing)

Lädt den vortrainierten STS-Checkpoint und wendet die geometrische Regularisierung (DN-Consistency, Normal Alignment) sowie Poisson Surface Reconstruction an. Dies ist der sensitivste Container bezüglich CUDA-Versionen.

Exkurs: Conda innerhalb von Docker

Obwohl Docker bereits das Betriebssystem isoliert und Conda somit theoretisch redundant wäre, wird Miniconda in diesem Container explizit genutzt. Der Grund: 3DGS-Repositories

(wie SuGaR) setzen stark auf vorkompilierte Conda-Binaries (z.B. `pytorch-cuda`) und exakte Paket-Versionen. Durch die strikte Nutzung von Conda im Docker-Container werden fatale C++ Kompilierungsfehler beim Rasterizer-Modul präventiv vermieden.

Requirements (Docker-Umgebung):

- **Base Image:** `nvidia/cuda:11.8.0-devel-ubuntu22.04` (Dieses Image bringt das kompatible CUDA 11.8 SDK direkt mit).
- **C++ Compiler:** Installation via `apt-get install build-essential` (installiert GCC/G++, was unter Linux Visual Studio ersetzt und garantiert mit CUDA 11.8 kompatibel ist).
- 24 GB VRAM (erforderlich, um in Paper-Evaluations-Qualität zu trainieren).

Installationsablauf (Dockerfile-Struktur):

```
# 1. Base Image mit CUDA 11.8 & GCC
FROM nvidia/cuda:11.8.0-devel-ubuntu22.04

# 2. System-Abhängigkeiten installieren
ENV DEBIAN_FRONTEND=noninteractive
RUN apt-get update && apt-get install -y \
    build-essential \
    git \
    wget \
    curl \
    libgl1-mesa-glx \
    libglib2.0-0 \
    && rm -rf /var/lib/apt/lists/*

# 3. Miniconda installieren
RUN wget https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh -O miniconda.sh
    bash miniconda.sh -b -p /opt/conda && \
    rm miniconda.sh
ENV PATH=/opt/conda/bin:$PATH

# 4. Conda Environment anlegen und Shell konfigurieren
RUN conda create --name sugar -y python=3.9
SHELL ["conda", "run", "-n", "sugar", "/bin/bash", "-c"]

# 5. PyTorch & CUDA 11.8 Bibliotheken installieren
RUN conda install pytorch==2.0.1 torchvision==0.15.2 torchaudio==2.0.2 \
    pytorch-cuda=11.8 -c pytorch -c nvidia -y && \
    conda install -c fvcore -c iopath -c conda-forge fvcore iopath -y && \
    conda install pytorch3d==0.7.4 -c pytorch3d -y && \
    conda install -c plotly plotly -y && \
    conda install -c conda-forge rich plyfile==0.8.1 jupyterlab nodejs ipywidgets -y

# 6. Repositories klonen und Submodule kompilieren
WORKDIR /workspace
```

```

RUN git clone https://github.com/Anttwo/SuGaR.git && \
  cd SuGaR && \
  pip install open3d PyMCubes && \
  pip install -e . && \
  cd gaussian_splatting/submodules/diff-gaussian-rasterization/ && \
  pip install -e . && \
  cd ../simple-knn/ && \
  pip install -e .

```

Conda-Umgebung im PATH systemweit als Standard setzen

```
ENV PATH=/opt/conda/envs/sugar/bin:$PATH
```

Details zur Build-Dauer und Kompilierungskomplexität (Container D):

In der praktischen Umsetzung stellte der Docker-Build für SuGaR eine signifikante Hürde dar, die mehrere Optimierungszyklen erforderte:

1. **Explizite CUDA-Architektur-Spezifikation (TORCH_CUDA_ARCH_LIST):** Um die Ausführbarkeit auf einer breiten Hardwarelandschaft (z.B. RTX 30er, 40er, Server-GPUs) abzusichern, mussten mehrere CUDA-Architekturen ("7.0;7.5;8.0;8.6;8.9;9.0") explizit beim Build hinterlegt werden. Der NVCC-Compiler kompiliert die C++ Extensions von `diff-gaussian-rasterization` und `simple-knn` für jede Architektur einzeln, was die Kompilierungszeit erheblich verlängert.
2. **Conda-Umgebung und Build-Isolation (--no-build-isolation):** Standardmäßig isoliert `pip` Builds in temporären virtuellen Umgebungen. Weil darin jedoch kein Zugriff auf die installierten PyTorch- und CUDA-Laufzeitbibliotheken von Conda besteht, kommt es zu fatalen Kompilierungsabbrüchen (z.B. bzgl. fehlender `torch`-Vektoren). Erst durch Hinzufügen von `--no-build-isolation` greift der Build-Prozess direkt auf den vorkonfigurierten Conda-Compiler-Stack zu.
3. **Parallelisierung via ninja-build:** Ohne eine parallele Kompilierungssteuerung fallen manche Python-Installer auf serielle Single-Thread-Kompilierung zurück, was zu extremen Timeouts oder Speicherüberläufen führt. Das Vorinstallieren des systemweiten Compilers `ninja-build` im Docker-Base-Image ermöglichte hierbei die maximal parallele Lastverteilung auf alle CPU-Kerne.

Maskenkonsistente SuGaR-Optimierung und objektbezogene Mesh-Extraktion

Ausgangsproblem und theoretische Konsequenz: Die Originalimplementierung von SuGaR verwendet für den photometrischen Rekonstruktionsverlust die vollständigen RGB-Trainingsbilder. Eine alleinige Filterung der 3DGS-Punktwolke auf Kabel-Gaussians reicht daher nicht aus: Der Optimierer würde ein objektisoliertes Modell gegen Pixel des gesamten Bildes – einschließlich Boden, Graben und Hintergrund – anpassen. Das erzeugt einen Zielkonflikt zwischen 3D-Initialisierung und Bildsupervision und kann trotz optisch überzeugender 3DGS-Renderings zu geometrisch falschen oder aufgeblähten Meshes führen. Deshalb werden Full-Scene-Baseline und Objektpfad strikt getrennt. Die Standarddatei `point_cloud.ply` bleibt eine vollständige Referenz; `point_cloud_filtered.ply` bewahrt die echte Filterung mit Originalopazitäten, während `point_cloud_filtered_opacity999999.ply` als bewusst geometrieorientierter Eingang in einen privaten maskenkonsistenten Objekt-Checkpoint kopiert wird.

Maskenhierarchie und Verluste: Für jede registrierte Kamera wird die SAM-3-Maske über den Bildnamen aufgelöst. Die aktuelle Datenprüfung bestätigte für alle 699 Kameras nichtleere Masken der Stufen `default`, `middle` und `small` bei einer Bildgröße von 768×432 Pixeln. Der RGB-Verlust wird nur auf Vordergrundpixeln $M(p) \in \{0, 1\}$ ausgewertet:

$$\mathcal{L}_{\text{RGB}}^M = \frac{\sum_p M(p) \ell_{\text{RGB}}(p)}{\sum_p M(p) + \varepsilon}.$$

Hierbei kombiniert ℓ_{RGB} den L1-Term mit DSSIM. Für die Depth-Normal Consistency wird die konservativere `middle`-Maske verwendet, um unsichere Silhouettenränder nicht als Normalenrestriktion zu interpretieren:

$$\mathcal{L}_{\text{DN}}^M = \frac{\sum_p M_{\text{middle}}(p) [1 - \mathbf{n}_{\text{depth}}(p)^\top \mathbf{n}_{\text{Gauss}}(p)]}{\sum_p M_{\text{middle}}(p) + \varepsilon}.$$

Beim anschließenden UV-Baking werden nur die von der jeweiligen Kamera als Objekt markierten Rasterpixel in die Texturakkumulation übernommen. Die `default`-Maske kann dabei um wenige Pixel dilatiert werden, damit geringe Silhouetten- oder Kalibrierungsungenauigkeiten keine sichtbaren Texturlücken erzeugen.

Gleiche Ausgangsmasken, unterschiedliche Wirkorte: Die SAM-Masken werden in der Pipeline mehrfach, jedoch mit unterschiedlicher Funktion verwendet. STS nutzt sie zunächst zur Objektzuordnung der Gaussians und erzeugt daraus `point_cloud_filtered.ply`; die Hochopazitätskopie `point_cloud_filtered_opacity999999.ply` dient anschließend als Standardinput für die Geometrieoptimierung. Dass diese Wolke in kamera-ausgerichteten Ansichten nahe an der Brille liegt, ist daher ein wichtiges Gegenindiz gegen grobe, frei schwebende Eingabe-Splats. SuGaR verwendet dieselben Ausgangsmasken anschließend erneut als Bildsupervision. Der aktuelle geführte Standard verwendet keine RGB- oder UV-Dilatation; solche Dilatationen bleiben getrennte Ablationen, weil unsichere Silhouetten- oder Hintergrundpixel die Geometrie indirekt beeinflussen können. Der DN-Term verwendet die einmal erodierte `middle`-Maske ohne zusätzliche Dilatation. Die UV-Maske begrenzt ausschließlich die spätere Farbsammlung im Texturatlas; sie ist kein geometrischer Loss und kann weder Coarse-Mesh noch verfeinerte PLY verändern.

Bedeutung der DN-Maske: DN steht für *Depth-Normal Consistency*. Die DN-Maske enthält keine gemessenen Tiefen oder Normalen und erzeugt auch keine Geometrie. Sie ist eine semantische Pixelgewichtung: `middle` ist die ursprüngliche SAM-Objektmaske nach einer 5×5 -Erosion und blendet damit den unsicheren Silhouettenrand aus. Für eine Trainingskamera rendert SuGaR eine Tiefenkarte und eine Normalenkarte aus seinen Gaussians. Aus lokalen Tiefendifferenzen wird eine zweite Normale abgeleitet; der DN-Loss bestraft ihre Abweichung von der gerenderten Gaussian-Normale nur innerhalb der `middle`-Maske und ohne zusätzliche Dilatation. Er nutzt also keine externe Normalenreferenz, sondern erzwingt Selbstkonsistenz der aktuellen Darstellung. Der Bildrand von einem Pixel wird technisch ebenfalls ausgeschlossen, weil dort keine Tiefennormale definiert ist. Im aktuellen Trainer wird dieser Term erst nach Zählerstand 9000 mit Gewicht 0.05 aktiviert. Die DN-Maske filtert weder die STS-Eingabewolke noch den UV-Atlas; im Refinement wird sie derzeit gar nicht verwendet.

Reproduzierbare Implementierung: Die Erweiterung liegt als versionsfixierter lokaler SuGaR-Fork unter `third_party/SuGaR` vor; der vollständige Ausgangscommit

ist zugleich im Dockerfile festgelegt. Während der Entwicklung überlagert die optionale Compose-Datei `docker-compose.sugar-dev.yml` diesen Quellstand in den bestehenden Containerpfad `/opt/sugar`; ein wiederholtes Herunterladen oder Neubauen des Images ist damit nicht erforderlich. Der Runner `run_masked_sugar.sh` erzeugt getrennte Ausgabeverzeichnisse, nutzt `dn_consistency` und führt die vollständige Kette aus Coarse-Optimierung, Mesh-Extraktion, Refinement und Textur-Export aus. Die geänderten Module wurden kompiliert; der maskierte RGB-Gradient sowie der maskierte Depth-Normal-Pfad wurden mit realen Kamera- und Maskendaten getestet. Ein vollständiger, zeitintensiver Objektlauf mit 699 Kameras hat inzwischen ein texturiertes OBJ erzeugt; seine geometrische Qualität ist jedoch noch Gegenstand der folgenden Stufenanalyse.

Entstehung des Coarse-Meshes: Der private Objekt-Checkpoint initialisiert SuGaR mit der aus der echten Filterung erzeugten Hochopazitätskopie der STS-Wolke. Die Originalopazitäten bleiben in `point_cloud_filtered.ply` für Diagnostik und Vergleich erhalten; die einheitliche hohe Eingangsoptazität ist ausschließlich eine Entscheidung für die geometrische Stützung der Standardroute. Im Coarse-Training werden gerenderte RGB-Bilder gegen die Originalbilder innerhalb der RGB-Maske optimiert. Die aktuelle lokale `dn_consistency`-Konfiguration übernimmt den Iterationszähler des geladenen STS-Modells und startet deshalb bei 6999. Die Angabe 15000 ist ein Zielwert dieses Zählers, nicht 15000 zusätzliche Optimierungsschritte; sie entspricht näherungsweise $15000 - 6999 = 8001$ Coarse-Updates. Zwischen Zählerstand 7000 und 9000 stabilisieren maskierter RGB-Loss und Entropieregularisierung die Darstellung. Nach 9000 werden zusätzlich die geometrischen SDF-Terme und die Depth-Normal Consistency aktiviert. Letztere vergleicht innerhalb der nicht dilatierten `middle`-Maske die aus der gerenderten Tiefe abgeleitete Flächennormale mit der Normalenrichtung der Gaussians:

$$\mathcal{L}_{\text{DN}}^M = \frac{\sum_p M_{\text{middle}}(p) [1 - \mathbf{n}_{\text{depth}}(p)^\top \mathbf{n}_{\text{Gauss}}(p)]}{\sum_p M_{\text{middle}}(p) + \varepsilon}.$$

Der verzögerte Start ist ein lokales Curriculum: Zunächst soll sich die photometrisch passende Dichteverteilung stabilisieren, bevor eine geometrische Normalenrestriktion auf möglicherweise noch unsichere Oberflächen wirkt. Ein Zielwert von 7500 ergäbe dadurch nur rund 501 neue Updates und noch keinen DN-Term; 12000 ergibt rund 5001 neue Updates, davon etwa 3000 nach Aktivierung der DN- und SDF-Terme. Die Untergrenze 9001 im Runner verhindert daher einen formal als `dn_consistency` bezeichneten, praktisch aber DN-freien Lauf. Diese Schwelle ist eine Implementierungseigenschaft des lokalen Trainers, keine allgemein bewiesene Mindestiterationszahl für jedes SuGaR-Projekt.

Die 699 registrierten Ansichten werden im aktuellen Lauf nicht zufällig geteilt. Mit `--eval True` legt der Wrapper jede achte Ansicht mit Index $i \bmod 8 = 0$ in die Testmenge. Das sind die Indizes $0, 8, \dots, 696$, also 88 Eval- und die übrigen 611 Trainingsansichten. Diese in SuGaR übliche Aufteilung reserviert Ansichten für eine spätere, von den Trainingsbildern getrennte Rendering-Evaluation; die Aufteilung selbst ist jedoch kein geometrischer Genauigkeitsnachweis. Coarse- und Refinement-Training übernehmen die Einstellung `--eval True`; die aktuelle Coarse-Mesh-Extraktion verwendet denselben Achtel-Split intern ebenfalls. Für einen kontrollierten Vergleich mit dem Referenzlauf bleibt dieser Split unverändert. Ein späterer All-View-Lauf mit allen 699 Ansichten wäre eine eigene Ablation und darf nicht mit dem RGB-Dilatationstest vermischt werden.

Nach dem Coarse-Training lädt die Mesh-Extraktion ausschließlich den Coarse-Checkpoint; sie liest keine 2D-Semantikmasken mehr ein. Zunächst werden Gaussians mit geringer

Opazität verworfen. Aus den Gaussian-Renderings der 611 Trainingsansichten werden Oberflächenpunkte und Normalen abgetastet. Der Referenzwert von 10 000 000 ist eine Zielzahl, die wegen begrenzter gültiger Samples pro Ansicht nicht zwingend exakt erreicht wird. Die erste ausgeführte Checkpoint-Ablation lieferte beispielsweise 9 424 416 Oberflächenpunkte. Auf diesem Punkt-Normalen-Satz führt Open3D die Poisson-Rekonstruktion aus. Die Poisson-Tiefe ist die Auflösung des zugrunde liegenden Oktrees: Ein höherer Wert erfasst feinere Strukturen, erhöht aber Laufzeit, Speicherbedarf und die Wahrscheinlichkeit, kleine fehlerhafte Flächen mit zu erhalten. Nach der Poisson-Rekonstruktion entfernt das aktuelle Dichtequantil die 10 % schwächsten Vertices, das Mesh wird auf das Vertexziel dezimiert und seine Vertices werden optional auf den nächsten abgetasteten Oberflächenpunkt projiziert. Damit ist die beobachtete erste Artefaktstufe mit dem gesamten Prozess aus Coarse-Regularisierung, Oberflächenabtastung, Poisson-Rekonstruktion, Dichtebereinigung, Decimation und Projektion vereinbar; noch keiner dieser Teilmechanismen ist einzeln als Ursache bewiesen.

Steuerung und ressourcenschonende Ablation: Der Runner `run_masked_sugar.sh` schützt sowohl den privaten Eingabe-Checkpoint als auch das Ausgabeverzeichnis eines Tags vor Überschreiben. Er unterstützt reproduzierbare Umgebungsvariablen und bei einem einfachen Terminalaufruf eine iterative Konfiguration mit dem Schlüsselwort **EXPLAIN**. Neben Maskenstufen, Dilatationen und Refinement lassen sich der Coarse-Zähler, das Vertexziel und die Zahl der Oberflächenstichproben dokumentiert wählen. Die Option `STOP_AFTER_COARSE_MESH=1` beendet einen neuen Lauf nach erfolgreichem Coarse-Training und Coarse-Mesh-Export; Refinement, UV-Baking und Crop werden dann bewusst ausgelassen. Dieser Tag ist damit absichtlich ein terminaler Coarse-only-Befund und wird nicht durch ein nachträgliches Umschalten auf 0 fortgesetzt. Für einen vollständigen Vergleich wird ein neuer Tag mit `STOP_AFTER_COARSE_MESH=0` gestartet. Dieser speichert zuerst das Coarse-PLY, liest es anschließend nur als Eingabe des Refinements und schreibt die verfeinerte PLY sowie das OBJ getrennt; das Coarse-PLY bleibt erhalten. Die neue Ablation `run_coarse_mesh_abl` lädt dagegen die vorhandene Datei `15000.pt` und erzeugt ausschließlich ein neues Coarse-Mesh-PLY. Sie führt weder Coarse-Training noch Refinement, UV-Baking oder Crop aus. Dadurch lassen sich Poisson-Tiefe, Dichtequantil, Oberflächenstichproben, Vertexprojektion und Decimation einzeln prüfen. Ein kleineres Vertexziel reduziert vor allem die Größe des ausgegebenen und anschließend zu verfeinernden Meshes; die kostspielige Oberflächenabtastung und Poisson-Rekonstruktion werden dadurch nicht im selben Verhältnis kürzer. Dafür ist die Zahl der Oberflächenstichproben ein direkter, aber qualitätsbehafteter Laufzeithebel.

Aktueller Befund und Stufenlokalisierung: Die gefilterte Objektwolke wurde in kamera-ausgerichteten Ansichten qualitativ gegen die Brillengeometrie geprüft. Sie zeigt keine auffälligen frei schwebenden oder groß abgelösten Splats; der visuell wahrgenommene Abstand liegt grob im Bereich bis etwa 1 mm. Diese Einschätzung ist weder geeicht noch metrisch referenziert und darf daher nicht als Genauigkeitsnachweis interpretiert werden. Die großflächigen Restflächen sind bereits im aus dem Coarse-SuGaR-Checkpoint extrahierten Mesh sichtbar und treten ebenfalls in der exportierten verfeinerten Gaussian-PLY auf. Die nicht unmittelbar visualisierbaren `.pt`-Dateien sind PyTorch-Checkpoints; für die vorliegende Lokalisation sind sie nicht erforderlich. Damit ist der UV-Textur- und OBJ-Export nicht die Primärursache. Gegenüber der visuell unauffälligen Eingabewolke grenzt der Befund die Ursache auf die Coarse-Regularisierung und/oder die anschließende Oberflächengewinnung ein, insbesondere Oberflächenabtastung, Poisson-Rekonstruktion,

Dichtebereinigung, Decimation und Projektion auf Oberflächenpunkte. Das anschließende Refinement übernimmt das Coarse-Mesh als Bindungsgeometrie und kann die vorhandenen Flächen daher erhalten oder verstärken; aus dem Befund folgt jedoch noch nicht, welcher dieser Teilschritte ursächlich ist. Die inzwischen ausgeführte Checkpoint-Ablation `depth8_v50000` ist ebenfalls ein Coarse-Mesh: Sie lädt den vorhandenen `15000.pt`-Checkpoint, verwendet Poisson-Tiefe 8 und dezimiert auf 50000 Vertices. In der ersten visuellen Inspektion wirkt dieses Mesh gegenüber dem Referenz-Coarse-Mesh deutlich geordneter und für eine schnelle Befundaufnahme brauchbar; die Restflächen sind jedoch weiterhin sichtbar. Da Poisson-Tiefe und Vertexziel gleichzeitig geändert wurden, belegt dieser Befund noch nicht, welcher der beiden Schritte die Verbesserung erzeugt. Für dünne Brillenelemente und eine spätere Auswertung ist 50000 voraussichtlich zu grob; ein Vergleich `depth8_v500000` trennt bei gleicher Poisson-Tiefe die höhere Meshauflösung von diesem ersten Screening-Befund.

Ein anschließender vollständiger Screening-Lauf kombinierte RGB- und UV-Dilatation 0 Pixel, Coarse-Zielzähler 12000, 100000 Mesh-Vertices, fünf Millionen Zielstichproben und das mittlere Refinement-Profil. Er war in ungefähr einer Stunde abgeschlossen. Sein Coarse-Mesh zeigt in der qualitativen Gegenüberstellung deutlich weniger Ausreißer und eine kompaktere Brillengeometrie als die vorherige `depth8_v50000`-Checkpoint-Ablation; einzelne Restflächen und Spitzen bestehen jedoch weiterhin. Auch die visualisierte verfeinerte Gaussian-Splat-PLY weist gegenüber dem vorherigen Refinement deutlich weniger großflächige Ausstrahlungen auf. Der nachfolgende Coarse-only-Kontrolllauf hielt Zielzähler, Vertexzahl, Stichprobenzahl, DN-Maske und Extraktion konstant und stellte ausschließlich die RGB-Dilatation von null auf zwei Pixel zurück. Sein Coarse-Mesh ist in der direkten visuellen Gegenüberstellung deutlich schlechter und enthält wieder große, ähnlich lokalisierte Restflächen. Damit ist die zusätzliche RGB-Randbeaufsichtigung als wesentlicher Artefakttreiber stark gestützt. Dass sich einzelne Spitzen zwischen den Läufen nicht exakt überdecken, ist wegen stochastischer Optimierung und Oberflächenabtastung erwartbar und schwächt den systematischen Unterschied der Flächen nicht. Die Beobachtung beweist nicht, dass Dilatation die einzige Ursache ist, weil auch der Null-Dilatationslauf weiterhin Restartefakte enthält.

Als nächste Maskenablation wird die RGB-Supervision vom undilatierten `default`-Rand auf die einmal erodierte `middle`-Maske umgestellt. “Eine Erosion” bezeichnet hier einen OpenCV-Durchlauf mit einem 5×5 -Kernel und damit ungefähr zwei Pixel Randabtrag pro Seite, nicht einen einzelnen Pixel. Über alle 699 Ansichten behält `middle` durchschnittlich 78,9% und im Median 80,7% der ursprünglichen Vordergrundfläche; keine Maske wird leer, in der stärksten Einzelreduktion verbleiben jedoch nur 42,0%. Deshalb wird `small` mit zwei Erosionsdurchläufen zunächst nicht getestet. Der vollständige `middle`-Lauf hält 12000, 100000, fünf Millionen, mittleres Refinement und null Dilatation konstant. Für die Geometrie ändert sich gegenüber dem bisherigen Null-Dilatationslauf nur `MASK_LEVEL=middle`; `TEXTURE_MASK_LEVEL=middle` betrifft ausschließlich die visuelle UV-Textur und wird zur konsistenten Silhouettendarstellung mitgeführt.

Die zugehörige verfeinerte PLY ist eine Gaussian-Splat-Datei mit 98281 Gaussian-Einträgen und enthält neben Position und Normale insbesondere SH-, Opazitäts-, Skalierungs- und Rotationsattribute, jedoch keine klassische Face-Liste. Sie ist daher nicht mit einer gewöhnlichen Mesh-PLY gleichzusetzen und wird von CloudCompare je nach Version nicht als Splat-Modell dargestellt. Das exportierte OBJ enthält dagegen 52385 geometrische

Vertices und 98281 konsistente Dreiecke; Materialdatei und PNG-Textur sind vorhanden. Der ursprüngliche Windows-UNC-Pfad erreicht jedoch exakt 260 Zeichen und damit die klassische `MAX_PATH`-Grenze mancher 3ds-Max-Importer. Zusätzlich fehlte dem von PyTorch3D geschriebenen OBJ ein abschließender Zeilenumbruch. Der lokale Exporter ergänzt diesen nun automatisch; für den bereits abgeschlossenen Lauf wurde unter `data/06_mesh/no_di` eine unveränderte Geometriekopie mit den kurzen Namen `mesh.obj`, `mesh.mtl` und `texture.png` erzeugt.

Nächste kontrollierte Prüfung: Zunächst werden die vorhandene Eingabe-PLY, das Coarse-Mesh, die verfeinerte PLY und das finale OBJ in denselben Kameraansichten protokolliert. Danach wird der vorhandene Coarse-Checkpoint zunächst ohne erneute Coarse-Optimierung oder Refinement für getrennte Mesh-Extraktionsvarianten wiederverwendet. Die Folgevergleichsvariante `depth8_v500000` hält die Poisson-Tiefe 8 des sichtbasiert besseren Screenings fest und erhöht nur das Vertexziel. Sie klärt, ob die grobe 50000-Decimation die Restflächen lediglich verdeckt oder ob der Befund bei einer brauchbaren Auflösung bestehen bleibt. Eine Variante ohne Vertexprojektion prüft dagegen die Rückprojektion auf die abgetasteten Oberflächenpunkte. Diese Ablationen ändern keine Masken und können deshalb die RGB-Dilatation nicht bewerten. Der anschließende RGB-Null-Dilatationslauf hält alle Referenzparameter konstant und setzt nur `MASK_DILATION_PX=0`; die UV-Dilatation bleibt für diesen kausalen Geometrievergleich beim Referenzwert. Mit einem neuen Tag und `STOP_AFTER_COARSE_MESH=0` erzeugt derselbe Lauf sowohl das erhaltene Coarse-PLY als auch die verfeinerte PLY und das texturierte OBJ. Die DN-middle-Maske, die Eingabewolke, Kameras, Coarse-Zähler, Stichprobenzahl und Vertexziel bleiben gleich. Der Multi-View-Crop wird dabei deaktiviert, da er die Ursache nicht lokalisiert und sich beim dichten Brillenmesh als zu konservativ beziehungsweise im `semantic-core`-Modus als zu destruktiv erwiesen hat. Der kombinierte Screening-Lauf mit Zielzähler 12000, 50000 Vertices und kurzem Refinement ist davon getrennt zu halten: Er verändert gleichzeitig die Zahl der Coarse-Updates, die dem Refinement übergebene Meshauflösung und die Zahl der Refinement-Updates. Mit reduzierter Stichprobenzahl käme eine vierte Veränderung hinzu. Er führt die gesamte Kette bis zum OBJ aus und überspringt nur den Crop; die RGB-Dilatation bleibt dabei, sofern nicht gesetzt, bei 2 Pixeln. Ein abweichendes Ergebnis kann deshalb weder der RGB-Dilatation noch einem einzelnen anderen Parameter zugeschrieben werden.

Multi-View-Konsens als geometrisches Quality-Gate: Eine einzelne 2D-Maske kann Okklusionen und lokale Fehlsegmentierungen nicht zuverlässig von Geometriefehlern unterscheiden. Daher wird das exportierte Mesh mit allen registrierten Kameras geraycastet. Für jede sichtbare Dreiecksfläche f wird ihr semantischer Unterstützungsanteil bestimmt als

$$s(f) = \frac{\#\{\text{sichtbare Kameras, die } f \text{ als Objekt unterstützen}\}}{\#\{\text{sichtbare Kameras, die } f \text{ beobachten}\}}.$$

Der konservative Modus bewahrt unterbeobachtete Flächen, um dünne oder nur einseitig sichtbare Strukturen nicht vorschnell zu löschen. Diese Vorsicht erwies sich bei einem sehr dichten Full-Scene-Testmesh jedoch als zu schwach wirksam: Bei 960 732 Dreiecken wurden bei einem Viertel der Originalauflösung nur 56 179 Flächen entfernt, weil der überwiegende Teil nicht ausreichend oft abgetastet wurde. Daraus folgt ein wichtiges Implementierungsergebnis: Die erforderliche Evidenz muss zur Dreiecksdichte und zur Kameraauflösung passen.

Der zusätzliche Modus `semantic-core` erhöht deshalb die Raycast-Auflösung auf die halbe Bildgröße, akzeptiert eine sichtbare Pixelbeobachtung als Evidenz und entfernt auch nicht beobachtete Flächen. Im aktuellen Kontrolllauf verblieben 6 277 semantisch unterstützte von 960 732 Flächen; 954 455 Flächen wurden entfernt. Dieser aggressive Export ist als visuell zu prüfender Objektkern zu verstehen, nicht als alleiniger Nachweis metrischer Genauigkeit. Für das dichte Brillenmesh ist er jedoch zu destruktiv und der konservative Modus zu schwach, um die Restflächen zuverlässig zu isolieren. Der Crop wird daher nicht als primäres Bereinigungsverfahren verwendet, sondern bleibt ein getrenntes semantisches Diagnosewerkzeug. Die konservative und die aggressive Variante werden separat gespeichert. Außerdem verdichtet der Export nur tatsächlich referenzierte Vertex- und UV-Einträge; dadurch sank die reine OBJ-Geometrie des Objektkerns von rund 111 MB auf rund 920 kB, während die Texturatlas-Datei bewusst unverändert erhalten bleibt.

Grenzen und wissenschaftliche Bewertung: Ein sinkender Trainingsverlust oder ein plausibles Gaussian-Splat-Rendering ist kein Beleg für eine korrekte vermessungstechnische Oberfläche. Die Wirksamkeit des maskenkonsistenten Pfades ist daher in der Bachelorarbeit über die aus dem Sub-Mesh extrahierte Centerline gegen GNSS-Referenzen zu prüfen. Insbesondere sind Abtragsrate, Vollständigkeit der Kabeloberfläche, RMSE und Hausdorff-Distanz gemeinsam zu berichten. Der Multi-View-Crop ist ein semantisches Sicherheitsnetz; er ersetzt weder eine hochwertige Kamerageometrie noch die spätere geometrische Evaluation.

[GELÖST: Nachträglich behobene Laufzeitfehler und interaktive Parameter-Erklärung, Stand 07/2026]

Nach initialer Fertigstellung des Containers traten beim ersten produktiven Pipeline-Lauf drei voneinander unabhängige Laufzeitfehler auf, die sukzessive behoben wurden. Zusätzlich wurde die Nutzerführung der Orchestrierungsskripte um einen Erklärungsmodus erweitert:

1. **Intel-MKL-Symbolkonflikt (`iJIT_NotifyEvent`):** Conda zieht bei der Installation von `pytorch-cuda=11.8` standardmäßig die aktuellste Intel-MKL-Bibliothek (Version ≥ 2025) nach. Ab dieser Version wurde jedoch das von PyTorch 2.0.1 intern benötigte Profiling-Symbol `iJIT_NotifyEvent` aus MKL entfernt, wodurch bereits der schlichte Python-Import von `torch` mit `ImportError: undefined symbol: iJIT_NotifyEvent` abstürzte. Die Lösung besteht in einer expliziten Versionsfixierung "`mk1==2024.0.0`" innerhalb der `conda install`-Transaktion, wodurch die letzte kompatible MKL-Generation erzwungen wird.
2. **Falscher Skript-Einstiegspunkt (`extract_mesh.py` statt `train.py`):** Der ursprüngliche Pipeline-Aufruf rief `extract_mesh.py -regularization dn_consistency` auf. Dieses Skript ist jedoch ausschließlich für die reine Mesh-Extraktion aus einem *bereits regularisierten* Coarse-SuGaR-Checkpoint zuständig und besitzt gar keinen `-regularization`-Parameter. Der Parameter `-r/-regularization_type` gehört stattdessen zum übergeordneten `train.py` im SuGaR-Wurzelverzeichnis, welches die komplette Kette (Coarse-Training mit Regularisierung $\rightarrow$ Mesh-Extraktion $\rightarrow$ Refinement) orchestriert. Die Pipeline-Skripte wurden entsprechend korrigiert, um `train.py` mit den Parametern `-s` (Szenenpfad), `-c` (STS-Checkpoint-Pfad) und `-i` (zu ladende Trainings-Iteration, passend zur STS-Iterationsanzahl aus Schritt 3) aufzurufen.
3. **Dateisystem-Berechtigungsfehler (`PermissionError: ./output`):** Da der Container aus Sicherheitsgründen nicht als `root`, sondern mit der Host-UID/GID des Nutzers

ausgeführt wird (`user: "${HOST_UID}:${HOST_GID}"` in der `docker-compose.yml`), schlug das Anlegen des Ausgabeverzeichnis `./output/coarse/...` fehl: Das Arbeitsverzeichnis `/opt/sugar` wurde während des Image-Builds vollständig als `root` angelegt (u. a. durch `git clone` und `pip install`) und ist für den nicht-privilegierten Laufzeitnutzer standardmäßig nicht beschreibbar. Die Lösung ist ein abschließend ausgeführter `RUN chmod -R 777 /opt/sugar`-Befehl im Dockerfile, der das gesamte SuGaR-Arbeitsverzeichnis für alle Nutzer beschreibbar macht, sodass zur Laufzeit erzeugte Unterordner (Checkpoints, Meshes) problemlos angelegt werden können.

4. **Verbindungspfad-Ladefehler bzgl. der Kameradaten (`cameras.json`):** Im originalen Quellcode von SuGaR (`sugar_scene/cameras.py`) ist das Laden der Kamerasystemkonfiguration fehlerhaft verknüpft, da der relative Pfad starr per einfacher String-Zusammenfassung (`gs_output_path + 'cameras.json'`) statt über eine sichere Pfadbibliothek verbunden wird. Ohne einen abschließenden Schrägstrich am Argumentenende des Pfades bricht die Ausführung unvermittelt mit einem `FileNotFoundError` für `/data/05_3dgs/outputcameras.json` ab, weil der Dateiname fälschlicherweise an den Ordernamen angehängt wird. Dieser systemische Fehler wurde gelöst, indem in allen Steuerungs- und Vorbereitungsskripten (`run_from_sts.sh`, `run_from_sugar.sh`) ein erzwungener, abschließender Schrägstrich an das Übergabeargument (`-c /data/05_3dgs/output/`) angehängt wird.
5. **Ephemerer Datenverlust bei Fertigstellung durch automatischen Container-Cleanup (`--rm`):** SuGaR schreibt sämtliche berechneten Intermediates (Coarse-Zwischenmodelle, die extrahierten geometrischen Oberflächennetze im `.ply`-Format und die finalen texturierten `.obj`-Modelle) standardmäßig unter relative Pfade des internen Arbeitsverzeichnis (`/opt/sugar/output/...`). Da die Container der Pipeline zur Ressourceneinsparung sequenziell über den Docker-Compose-Parameter `--rm` ausgeführt und nach Beendigung rückstandslos gelöscht werden, gingen alle Mesh-Modelle ephemeral unmittelbar nach der Generierung verloren und waren auf dem Host-System nicht auffindbar. Dieser schwerwiegende Datenverlust wurde strukturell gelöst, indem das Ausgabeverzeichnis von SuGaR dynamisch als persistentes Host-Volume über die `docker-compose.yml` gemountet wird (`- ./data/sugar_output:/opt/sugar/output`). Dies stellt sicher, dass alle berechneten CAD-Daten physisch aus der Kapselung des Containers auf die nutzbare Partition des Host-Systems unter `data/sugar_output/` ausgelagert werden.
6. **Interaktive Parameter-Erklärung ("EXPLAIN"-Modus):** Um die wissenschaftliche Nachvollziehbarkeit der Parameterwahl auch für Nicht-Experten zu gewährleisten, wurde die interaktive Konfigurationsabfrage der Orchestrierungsskripte (`run_from_sts.sh`, `run_from_sugar.sh`) um einen Erklärungsmodus erweitert. Anstelle eines direkten Parameterwerts kann der Nutzer bei den Abfragen für Regularisierungstyp und Refinement-Dauer das Schlüsselwort `EXPLAIN` eingeben. Das Skript gibt daraufhin eine allgemeinverständliche Erläuterung aller verfügbaren Optionen aus (z. B. den methodischen Unterschied zwischen `dn_consistency`, `density` und `sdf`, bzw. den Rechenzeit-/Qualitäts-Kompromiss zwischen `short`, `medium` und `long`) und wiederholt anschließend dieselbe Abfrage, ohne die Pipeline abzuberechnen.
7. **Automatisierte OCR-Erfassung der Transformationsmatrix (Tesseract):** Da das fehlerfreie, manuelle Abtippen der 16 hochpräzisen Gleitkommazahlen der 4x4-Transformationsmatrix aus der CloudCompare-Konsole fehleranfällig und mühsam

ist, wurde eine automatisierte optische Zeichenerkennung (OCR) integriert. In Container A (SAM 3 Preprocessing, Python 3.12) wurden die Systemabhängigkeit `tesseract-ocr` und die Python-Schnittstelle `pytesseract` nachgerüstet. Der Breakpoint des Master-Skripts fragt nun interaktiv: *"Möchten Sie die Matrix per OCR einlesen?"*. Der Nutzer muss lediglich einen Screenshot des Matrixfeldes unter `data/01_raw/matrix_screenshot.` ablegen. Das Skript `ocr_matrix.py` bereinigt OCR-Ungenauigkeiten (wie z.B. das Verschlucken von Dezimalpunkten oder unvollständige Vorzeichen) vollautomatisch und speichert das Ergebnis formatiert als CSV-Matrix unter `data/04_sfM/matrix.txt` ab. Erst nach erfolgreicher Sichtprüfung und Bestätigung des Nutzers fährt die Pipeline fort.

8. **Entwicklung einer interaktiven Web-Steuerungskonsole (Web-UI):** Um die Pipeline für Fachfremde und Ingenieure anwenderfreundlicher zu gestalten, wurde ein interaktives Web-Dashboard entwickelt, welches die Orchestrierung via Browser ermöglicht. Die Umsetzung stützt sich auf eine asynchrone Multi-Threaded HTTP-Server-Architektur, bei welcher die Shell-Befehle über ein virtuelles Linux-Pseudoterminal (`pty`) ausgeführt werden. Dies löst das Problem unvollständiger Pufferung: CLI-Prompts ohne Zeilenumbruch (z. B. interaktive Abfragen der Skripte) werden latenzfrei und in Echtzeit per Server-Sent-Events (SSE) gestreamt. Zusätzlich verfügt die Webkonsole über einen Drag-&-Drop-Uploader zur Dateisystem-Schnittstelle (Upload von Drohnenvideos oder des CloudCompare-Matrix-Screenshots) und schützt wertvolle VRAM-Ressourcen, indem beim Schließen oder Verlassen des Browser-Tabs alle asynchron im Hintergrund laufenden GPU-Prozesse automatisch abgebrochen werden (`sendBeacon`-Signalisierung). Befände sich der Nutzer mitten in einem Trainingslauf und lädt die Seite lediglich neu (Browser-Refresh), koppelt sich das Frontend dank einer prozessunabhängigen Terminal-Historie vollautomatisch und lückenlos zurück an die laufende Inferenz.
9. **Robuster, dateibasierter Pre-Run Guard für geodätische GCP-Passpunktdaten:** Da das einwandfreie Berechnen der lokalen Transformationsmatrix eine gültige Liste globaler GNSS-Passpunkte voraussetzt, wurde ein automatisierter, dateibasierter Guard am Einstiegspunkt der Pipeline (Schritt 0) integriert. Das Orchestrierungsskript prüft nun vollautomatisch das Vorhandensein der globalen GCP-Koordinatendatei (`gcp_coordinates.csv`) oder bereits berechneter relativer Transformationsdateien (`gcp_relative.csv`). Fehlen diese Daten gänzlich, pausiert die Ausführung an einem definierten Breakpoint und fordert den Anwender über das Terminal bzw. die Web-UI auf, die GNSS-Referenzdaten bequem per Drag & Drop im Dashboard im Verzeichnis `data/01_raw/` bereitzustellen. Erst nach detektiertem Upload und Bestätigung wird die relative GNSS-Projektion asynchron fortgesetzt, um vermeidbare Folgeabstürze im SfM- und CAD-Schnittstellenbereich präventiv zu eliminieren.

Automatisierte Punktwolkenfilterung zur Rauschminimierung und Qualitätssicherung (`filter_cable_pc.py`):

Die Filterung adressiert Hintergrund-Splats, nahezu unsichtbare „Dampf-Splats“ und schwarze Rauschartefakte (Floating Artifacts bzw. Rauch-Gase). Da die Originalversion von SuGaR keine vorab berechneten Objektmasken in den RGB-Verlust einbezieht, darf die gefilterte Objektwolke jedoch nicht unbemerkt die Standard-Checkpoint-Datei ersetzen. Das Hilfsprogramm erzeugt deshalb einen separaten Objekt-Export; dieser wird nur im maskenkonsistenten SuGaR-Pfad verwendet. Es führt eine dreidimensionale Qualitätsfilterung durch:

- **1. Objektspezifische Selektion (Vordergrund-Isolierung):** Das Skript parst den binären PLY-Header des STS-Szenen-Outputs und liest die zugewiesenen Segmentierungs-Vektoren aus. Zu jedem 3D-Gaussian existieren die durch STS gelernten Objekt-IDs (`obj_id_s`, `obj_id_m`, `obj_id_l`). Über die Parameter `--level m` und eine datensatzspezifisch geprüfte `--object_id` extrahiert das Skript die Kabel-Vordergrund-Splats; im aktuellen Versuch ist dies `object_id 0`. Unkorrelierte Hintergrundpunkte werden nicht in den Objekt-Export übernommen.
- **2. Opazitäts-Filterung (Eliminierung transparenter Floater):** Da während der mathematischen Optimierung viele fragile „Mogel-Splats“ mit extrem geringer Dichte entstehen, wird die Logit-Opazität α_{logit} des Punktes über die logistische Sigmoid-Funktion in eine echte physische Transparenzwirkung überführt:

$$\alpha_{\text{sigmoid}} = \frac{1}{1 + \exp(-\alpha_{\text{logit}})}$$

Punkte, deren berechnete $\alpha_{\text{sigmoid}} < 0.01$ beträgt, werden als unsichtbares Rauschen klassifiziert und vollständig verworfen.

- **3. Farbschwellenwert-Filterung (Entfernung schwarzer Dunstwolken):** 3DGS speichert Farbinformationen blickrichtungsabhängig in Form von sphärischen Harmonischen (SH). Die Koeffizienten der 0. Ordnung (f_dc_0 , f_dc_1 , f_dc_2) repräsentieren den diffusen Farbanteil. Um schwarze Splats (die das visuelle Bild stören und durch Rekonstruktionsunschärfen entstehen) mathematisch zu identifizieren, werden sie in den linearen RGB-Farbraum transformiert:

$$RGB = 0.5 + 0.28209479177387814 \cdot [f_dc_0, f_dc_1, f_dc_2]^T$$

Punkte, bei welchen sämtliche Farbkanäle unter einem einstellbaren Schwellenwert (Standard ≤ 0.08) liegen, werden zuverlässig als schwarzer Ruß oder Dunst identifiziert und aus der Geometrie eliminiert.

- **4. Dynamisches Quality-Gate (Sicherheits-Fallback):** Da ein totaler Datenverlust die nachfolgenden Meshing-Schritte sabotieren würde (SuGaR benötigt für die Oberflächenrekonstruktion eine Mindestanzahl an Stützpunkten), ist eine automatische Schutzlogik eingebaut. Sollte die Kombination aller Filterstufen fälschlicherweise 0 Punkte zurückliefern, verwirft das Skript die strengen Opazitäts- und Farbausschlüsse automatisch und reicht die rohe Objektsegmentierung als stabiles Fallback weiter.

Durch diesen erweiterten, mehrstufigen Qualitätsfilter-Lauf wird wirksam verhindert, dass SuGaR unvollständige oder durch schwarze Partikelblasen gestörte Oberflächen extrahiert, was die geometrische Präzision des finalen CAD-Modells drastisch verbessert.

Wichtige Best-Practices & Parameter-Tuning (SuGaR):

Für qualitativ hochwertige Rekonstruktionen (insb. dünne Kabelstrukturen) sind folgende Parameter entscheidend (siehe SuGaR Tips):

- **Regularisierung:** Nutze zwingend `-r "dn_consistency"` für die beste Mesh-Qualität.
- **Löcher im Mesh:** Wenn die Mesh-Bereinigung zu aggressiv ist, setze `vertices_density_quantile = 0.` in `sugar_extractors/coarse_mesh.py`.
- **Ellipsoide Artefakte (Bumps):** Reduziere die Poisson-Tiefe `poisson_depth` (z.B. von 10 auf 7 oder 8 in `coarse_mesh.py`), falls einzelne Gaussians sichtbar werden.
- **Große Szenen:** Bei weiten Distanzen sollten Custom Bounding Boxes (`-bboxmin` und `-bboxmax`) genutzt werden, um die Extraktion zu fokussieren.

Container E: DGtal & GDAL (Post-Processing)

Führt die Extraktion der Centerline auf dem isolierten Kabel-Sub-Mesh durch und überführt diese in georeferenzierte GIS-Formate (GeoJSON). Da DGtal eine komplexe C++ Bibliothek mit zahlreichen Abhängigkeiten (Eigen, CGAL, ITK) ist, wird direkt auf dem offiziellen `dgital:latest` Docker-Image aufgebaut.

Requirements & Repositories:

- **DGtal Release (v2.1):** <https://github.com/DGtal-team/DGtal/releases/tag/v2.1>
- **GDAL:** <https://github.com/OSGeo/gdal>
- **OGR2OGR Wrapper:** <https://github.com/wavded/ogr2ogr>

Docker Build & Ausführung:

```
# Offizielles DGtal Image bauen (im DGtal Repo Ordner)
docker build -t dgital:latest .
```

```
# Nachinstallation von GDAL (ogr2ogr) und Python-Paketen über ein abgeleitetes Docker-Image
FROM dgital:latest
USER root
RUN apt-get update && apt-get install -y gdal-bin libgdal-dev python3-pip python3-numpy
rm -rf /var/lib/apt/lists/*
USER digital
```

```
# Interaktive Ausführung mit Bind-Mount
docker run -it -v /host/path/data:/data --user=digital dgital-gdal:latest bash
cd /home/digital/git/DGtal/build/examples
# 1. DGtal Centerline-Extraktion auf dem lokalen Kabel-Mesh ausführen
# (Erzeugt /data/centerline_local.csv)
```

```
# 2. Ausführen des Python-Skripts zur Georeferenzierung
# (Wendet 4x4-Transformationsmatrix an und addiert UTM-Koordinaten des Anker-GCPs automatisch)
python /data/scripts/transform_centerline.py \
    --input_csv /data/centerline_local.csv \
    --matrix /data/matrix.txt \
    --anchor_txt /data/01_raw/anchor.txt \
    --output_csv /data/centerline_utm.csv
```

```
# 3. Konvertierung des UTM-Ergebnisses in standardisiertes GeoJSON (EPSG:25832)
ogr2ogr -f "GeoJSON" /data/centerline.geojson /data/centerline_utm.csv -a_srs EPSG:25832
```

Ergebnis der middle-Ablation und Konsequenz für das Endprodukt

Die visuelle Gegenüberstellung des vollständigen middle-Laufs mit dem ansonsten vergleichbaren default-Lauf bei null RGB-Dilatation ergibt keinen eindeutigen Sieger. Die einmal erodierte Maske reduziert die weit von der Sonnenbrille entfernten Restflächen und damit die deutlichsten frei liegenden Artefakte. Gleichzeitig werden die Objektgrenzen unruhiger

beziehungsweise zackiger, es treten zusätzliche lokale Randartefakte auf und in einzelnen Gläsern sind kleine Löcher sichtbar. Der `default + 0px`-Lauf erhält die Brillengeometrie an den Rändern und die Glasflächen besser, enthält dafür mehr abgelöste Flächen im Umfeld. Die zugehörige importerfreundliche Kopie des `middle`-Ergebnisses liegt unter `data/06_mesh/middle_no_dilation_screen_import`.

Dieser Befund spricht dafür, dass die verbleibende Schwierigkeit nicht mehr ausschließlich als Maskenfehler beschrieben werden kann. Eine binäre Bildmaske gewichtet einzelne Pixel, während ein Gaussian als zusammenhängende dreidimensionale Primitive mit räumlicher Ausdehnung optimiert und später in der Mesh-Extraktion berücksichtigt wird. Liegt der projizierte Gaussian teilweise innerhalb und teilweise außerhalb der erodierten Maske, entsteht eine diskrete Einschlussentscheidung; das Primitive wird nicht anteilig geometrisch geteilt. Dadurch kann die Erosion zwar unsichere Hintergrundsupervision vermindern, aber zugleich echte Randstützen entfernen oder die lokale Oberflächenbildung destabilisieren. Die einzelnen Löcher und die zackigeren Silhouetten sind mit dieser Erklärung vereinbar. Sie beweisen jedoch nicht, dass ausschließlich die Gaussian-Inklusion verantwortlich ist; Coarse-Regularisierung, Poisson-Rekonstruktion, Dichtebereinigung und das an das Coarse-Mesh gebundene Refinement bleiben mögliche Mitverursacher.

Für das Ziel dieser Projekt- und Bachelorarbeit ist deshalb nicht automatisch die Variante mit den wenigsten sichtbaren Artefakten die beste. Gegenstand ist keine vollständig metrologisch wahre Rekonstruktion jedes Details der Sonnenbrille, sondern ein korrekt vermessenes, georeferenziertes und wie eine bereinigte Punktwolke nutzbares Produkt. Eine geometrische Veränderung durch einen nachgelagerten Bereinigungsschritt ist unter diesem Ziel zulässig, sofern sie als Postprocessing transparent dokumentiert, reproduzierbar ausgeführt und im georeferenzierten Raum gegen die projektbezogenen Referenzen geprüft wird. Für den angesetzten Toleranzrahmen von etwa ± 10 cm sind insbesondere die Lage der relevanten Objektteile, die Vollständigkeit der Nutzgeometrie und das Ausbleiben größerer Ausreißer zu bewerten; ein optisch glatterer Rand allein ist kein Genauigkeitsnachweis.

Als aktuelle Arbeitshypothese wird daher `default + 0px` als geometrische Basis bevorzugt. Die vorhandene Rand- und Glasgeometrie ist wertvoller als eine aggressive Entfernung, weil fehlende Randbereiche oder Löcher durch ein späteres Cleaning nicht zuverlässig rekonstruiert werden können. Die weiter entfernten Artefakte sind dagegen als eigenständige, geometrisch nicht zum Objekt gehörende Flächen prinzipiell gezielter entfernbar. Der `middle`-Lauf bleibt eine wichtige Vergleichsvariante und kann dann die bessere Produktbasis sein, wenn die entfernten Artefakte die spätere Auswertung stärker beeinträchtigen als die zusätzlichen Randverluste. Diese Entscheidung wird nicht nur visuell, sondern anhand derselben metrischen Qualitätskriterien getroffen.

Der nächste sinnvolle Schritt ist ein getrennt gespeicherter Mesh-Bereinigungstest auf der bevorzugten Basis. Der lokale SuGaR-Pfad stellt bereits ein optionales `postprocess_mesh` bereit, das iterativ schwach dichte Randdreiecke anhand der Mesh-Konnektivität entfernt und anschließend ausgewählte Dreiecke mit ausreichender Dichte wieder zulässt. Diese Funktion wurde in den beiden bisherigen Läufen noch nicht als Lösung aktiviert. Sie darf nicht blind mit den Standardwerten eingesetzt werden, weil sie ebenfalls Löcher oder relevante dünne Strukturen entfernen kann. Stattdessen werden das unveränderte `default + 0px`-Mesh, eine konservative Postprocessing-Variante und der unveränderte `middle + 0px`-Vergleich separat gespeichert. Bewertet werden die Entfernung eindeutig abgelöster Flächen, die Randstabilität, die Anzahl beziehungsweise Lage der Glaslöcher

und die Abweichung der georeferenzierten Referenzpunkte. Das unveränderte Mesh bleibt als wissenschaftlicher Ausgangsbefund erhalten; das bereinigte Mesh wird als abgeleitetes Produkt gekennzeichnet.

Iterations- und Opazitätsbefund an kontaktkritischen Bereichen

Eine weitere kontrollierte Serie verwendet durchgehend die undilatierte `default`-RGB-Maske, die `middle`-DN-Maske, fünf Millionen Oberflächenstichproben, ein Ziel von 200000 Mesh-Vertices, mittleres Refinement und keinen nachgeschalteten Crop. Variiert wird nur der Coarse-Zielzähler. In der qualitativen Gegenüberstellung erzeugt der Lauf bis 18000 gegenüber 9001 und 10000 deutlich geschlossener und konsistentere Brillenbügel. Die zuvor beobachteten Restflächen verschwinden jedoch nicht vollständig, sondern treten an weitgehend denselben Kontakt- beziehungsweise Ansatzbereichen zwischen Sonnenbrille und Untergrund wieder auf. Ihre Lage und Ausdehnung wirken bei der längeren Optimierung stabiler. Der Befund ist daher nicht als zufälliges Rauschen zu interpretieren, aber auch noch kein Beweis dafür, dass Poisson allein die Flächen erzeugt.

Die Checkpoints stützen diese Einordnung quantitativ. Der lokale Trainer startet bei Zählerstand 6999 und führt bei 9001 vor der Berechnung des Bildverlusts die harte Bereinigung aller Gaussians mit $\alpha < 0,5$ aus. Der 9001-Checkpoint enthält danach 2363 Gaussians, der 10000-Checkpoint 2368; in beiden liegen alle gespeicherten Opazitäten oberhalb von 0,5. Der 18000-Checkpoint enthält ebenfalls 2368 Gaussians, aber nur 1358 davon liegen noch oberhalb von 0,5; 1010 Gaussians liegen darunter. Der Mesh-Extractor wendet vor der kamerabasierten Oberflächenabtastung dieselbe feste Schwelle $\alpha > 0,5$ erneut an. Damit nimmt er beim 18000-Lauf nur diese 1358 stark opaken Gaussians in die Oberflächengewinnung auf. Die resultierenden Coarse-Meshes besitzen dennoch vergleichbare Größen: 98891/190988 Vertices/Dreiecke bei 9001, 101242/192670 bei 10000 und 101559/195771 bei 18000. Die refined PLY enthält jeweils genau einen Gaussian pro Coarse-Dreieck.

Der 9001-Lauf ist dabei kein aussagekräftiger vollständiger DN-/SDF-Lauf: Die Bereinigung tritt bei 9001 ein und erst dieser eine letzte Update-Schritt aktiviert DN- und SDF-Terme. 10000 enthält etwa 1000, 18000 etwa 9000 Updates nach ihrer Aktivierung. Daraus folgt als Arbeitshypothese: Die längere geometrische Optimierung kann die Positionen, Skalen, Orientierungen und die photometrisch relevante Dichte der starken Oberflächenstützen so ordnen, dass dünne Bügel geschlossener wirken. Dieselbe Optimierung kann jedoch auch wiederkehrende, mehransichtig nicht eindeutig getrennte Kontaktbereiche konsolidieren. Die vollständige Opazitätsdiagnose der gefilterten 6175 STS-Gaussians zeigt keine frei abgelösten Eingangsausreißer. Sie schließt aber nicht aus, dass an den Kontaktbereichen räumlich ausgedehnte Gaussians, unvollständige Vordergrund-Hintergrund-Trennung, unsichere Normalen oder die anschließende Poisson-Flächenschließung gemeinsam eine stabile falsche Fläche hervorbringen.

Die zunächst gestartete Extractor-Variante mit gesetztem Shell-Parameter `LOW_OPACITY_GAUSSIAN_` darf noch nicht als negativer Opazitätsbefund interpretiert werden. Zum Zeitpunkt dieses Laufs wurde die Umgebungsvariable im Ablationsskript weder eingelesen noch an `extract_mesh.py` weitergereicht; die lokale Extractor-Konstante blieb deshalb bei $\alpha > 0,5$. Die nahezu identische Geometrie war folglich erwartbar, weil effektiv die Referenzextraktion wiederholt wurde. Der Parameter ist inzwischen vom Ablationsskript bis zur Prune-Funktion durchgängig

implementiert, wird protokolliert und akzeptiert Werte in $[0, 1)$. Ein erneuter Test muss wegen des bereits belegten Ausgabeordners mit einem neuen Ablationstag erfolgen. Erst der Vergleich $\alpha > 0,5$ gegen $\alpha > 0$ bei ansonsten unveränderten Einstellungen kann entscheiden, ob schwach opake *Coarse*-Gaussians für die Oberflächenabtastung relevant sind.

Von dieser Extractor-Schwelle ist die konzeptionell andere Eingangsoptizität zu trennen. Die historische Datei `point_cloud_cable_opacity999999_diagnostic.ply` war kein Filter: Ihr Erzeuger erhielt alle Vertexdatensätze und überschrieb nur jedes gespeicherte Optizitätslogit mit einem einheitlichen Wert, der $\alpha \approx 0,999999$ ergibt. Die aktive Vorbereitung führt diese Überschreibung erst nach der echten Objekt- und Qualitätsfilterung aus und schreibt sie als `point_cloud_filtered_opacity999999.ply`. Damit ist die Hochoptizitätsdatei die Standardinitialisierung für den geometrieorientierten mask-aware SuGaR-Pfad, aber keine Auswahl besonders verlässlicher Splats und keine physikalische Transparenzmessung. Auch bei 9001 erfolgen kamerabasierte Oberflächenabtastung und Poisson-Rekonstruktion; aus der Eingangsoptizität folgt daher keine direkte Eins-zu-eins-Umwandlung der Splats in ein Mesh.

Ein tatsächlicher Eingabe-Optizitätsfilter wird deshalb separat mit `filter_cable_pc.py -min_opacity` erzeugt. Gegenüberzustellen sind die bisherige Objektwolke mit Mindestoptizität 0,01 und 6175 Gaussians sowie die aus derselben vollständigen STS-Wolke erzeugten strengeren Schwellen 0,25 und 0,5. Von 38196 semantisch zugeordneten Ausgangs-Gaussians verbleiben nach derselben Farbbereinigung 2985 beziehungsweise 2225; die Schwellen sind also starke Eingriffe und müssen vor dem Training insbesondere auf Glas- und Bügelstützen geprüft werden. Die aktuelle Standardroute verwendet dagegen die echte Filterung mit 0,01 und schreibt danach die getrennte Hochoptizitätskopie für SuGaR. Diese Kopie ist damit ein Kandidat für den wissenschaftlichen Produktpfad, jedoch weiterhin nur als geometrieorientierte Initialisierung zu interpretieren. Falls auch eine echte Eingabeschwelle die Kontaktflächen nicht reduziert, liegt der stärkste verbleibende Hebel nicht mehr in der Optizität, sondern in der Aufnahme- und Beobachtungsgeometrie: zusätzliche seitliche und rückwärtige Ansichten, eine klare räumliche Trennung vom Untergrund sowie diffuse, reflexionsarme Beleuchtung. Die spiegelnden beziehungsweise transparenten Gläser liefern nur schwache photometrische Korrespondenzen; ihre vollständig metrische Rohrekonstruktion ist daher mit dem vorliegenden Bildsatz grundsätzlich begrenzt und sollte gegebenenfalls als nachgelagert modellierte, klar deklarierte Nutzgeometrie behandelt werden.